Highlights

One Port for Many Protocols: The Cost, Robustness and Mechanism of First-Bytes Demultiplexing

Alex I. Tsvetanov

- Host demultiplexing decides the protocol on the accepting machine.
- One shared listener is one kernel socket, not one per protocol.
- Passing cost ratios stay inside a pre-registered margin.
- Some of those intervals exclude a ratio of one.
- sslh, HAProxy, Envoy filters, ALPN and SO_REUSEPORT.

One Port for Many Protocols: The Cost, Robustness and Mechanism of First-Bytes Demultiplexing

Alex I. Tsvetanov[a]

[a]*Faculty of Computer Systems and Technologies, Technical University of Sofia, ul. Professor Georgi Bradistilov 11, Sofia, Bulgaria. ORCID 0009-0001-8369-150X.*

Abstract

This paper measures host demultiplexing, the protocol decision on the machine that accepted the connection. If the first bytes on one transport are disjoint, that host needs one listening descriptor per transport and accept model, not one per protocol. We built `oneport`, a minimal C++23 server for HTTP/1.1, h2c, TLS, SSH and MQTT on epoll, io_uring and I/O completion ports. We compared one listener with one port per protocol on two machines, Linux and Windows. Under load of one protocol, one-port mode was equivalent within 0.98 to 1.02 in 30 of the 32 tested cost cells. All three cost tests passed for HTTP/1.1, TLS and MQTT on Linux and for HTTP/1.1 and TLS on Windows. Those TLS passes have little power to show the cost of the byte check, since the handshake dominates. The passing cells stayed inside 0.98 to 1.02 of dedicated mode. The interval [0.9900, 0.9933] for HTTP/1.1 churn on epoll excludes one. On Linux, h2c's latency at a fixed load was not shown equivalent, once as a measured cost. The detector decided all 26,240 hard-case runs as specified. In that test, nginx, HAProxy, Envoy and hyper-util held at least 1.10 times the server's counted footprint per pending connection; sslh-ev held less. On epoll the kernel count is 1 for the shared listener and 6 in dedicated mode, with no load. Section 3.3 gives the other rows. A mixed-protocol load starved new connections on the io_uring backend in both modes.

Keywords: protocol detection, demultiplexing, TLS, MQTT, epoll, benchmarking, robustness

1. Introduction

A network service that speaks several protocols usually listens on one port per protocol. Each port is a socket of its own, a rule of its own in every firewall on the path, and a number every client must be told. The alternative is one

Email address: `atsvetanov@tu-sofia.bg` (Alex I. Tsvetanov)

[1][For Alex: the corresponding author's telephone number, which Computer Communications asks for at submission.]

listening socket for all of them. The server reads the first bytes a client sends and decides the protocol of the connection from them. We call this first-bytes demultiplexing.

This paper studies host demultiplexing. The protocol is decided on the machine that accepted the connection. The practice that decision explains includes sslh, HAProxy and Envoy's listener filters. ALPN negotiates the application protocol inside one TLS port. `SO_REUSEPORT` selects a socket before any application byte exists (Section 2). Section 3.3 states how many listening descriptors that decision needs, and checks the count by asking the kernel.

Many systems already do it. Proxies such as nginx, HAProxy, Envoy, caddy-l4 and sslh choose a route from the first bytes of a connection, before a back end sees it [1, 2, 3, 4, 5]. Server libraries such as Netty, Jetty, cmux and hyper-util choose a protocol handler inside one process [6, 7, 8, 9]. Standards leave room for it. A client of HTTP/2 with prior knowledge opens with a fixed preface [10]. A TLS client opens with a ClientHello [11]. A proxy that prepends a PROXY header does so on a listener configured to expect it [12].

What the decision costs is not known. In the documentation and source code we read, no project publishes a measured cost of its detection; at most, a project calls the cost small. We found no peer-reviewed paper whose subject is a server that serves several protocols on one listener (Section 2). Three questions follow.

- **Cost.** Under load of one protocol at a time, does a listener that detects the protocol cost more than the same server with one dedicated port per protocol? We ask it for throughput with a new connection per exchange, for requests per second on kept-alive connections, and for latency at a fixed load.

- **Robustness.** Does the detector decide a fixed table of hard cases as specified? Are its timed events bounded? Does it hold less memory per pending connection than the proxies and libraries that detect protocols today?

- **Mechanism.** Which internal choices matter? We compare leaving the first bytes in the kernel (peek) with reading them once and replaying them. We compare handing a connection to an in-process handler with relaying it to a back end. And we compare the server's relay with each proxy.

We answer these questions for one server, `oneport`, built for this paper. The answers are claims about this server, on two hosts, under stated conditions; they are not claims about every server. The hypotheses, their tests, the rules of validity and the order of every run were written and frozen on 2 October 2026, before any code of the server existed. Changes after that date went only into a dated revision log.

The paper contributes four things, each built and tested for the purpose:

1. A minimal C++23 server with one listening socket per process, for HTTP/1.1, HTTP/2 over cleartext with prior knowledge (h2c), TLS,

SSH and MQTT. Its detector is a table of `constexpr` matchers. Checks at compile time confirm that the classes are prefix-disjoint over a corpus taken from the specifications, so the order of the table cannot change a decision on it. A PROXY header is parsed per listener, and one server-speaks-first protocol can be reached through a timed fallback.

2. Two detection modes (replay and peek) and two dispatch modes (in-process, and relay with TLS passed through by its ClientHello). They run on three backends: epoll and io_uring on Linux and I/O completion ports (IOCP) on Windows. The same binary runs a dedicated mode with one port per protocol and no detection. That mode is the baseline of the cost questions.
3. A pre-specified evaluation in three families. Cost is tested for equivalence within a ratio of 0.98 to 1.02. Robustness has a frozen table of 25 hard cases, bounds on every timed event, and a test of the memory held per pending connection against five proxies and four libraries. The mechanisms get superiority tests. Every tested cell needs both a clustered bootstrap interval and an exact sign test to pass. The number of sessions of the cost family is sized from an A/A pilot before any comparison of the two modes runs.
4. Load and case generators, a holder that measures the kernel's memory for a bare socket, and pinned configurations of every competitor. Every measured build is gated on green sanitizer records of its first-party code.

Section 2 reviews the protocols' first bytes, the systems that detect them, and related research. Section 3 describes the server. Section 4 gives the hypotheses and the method as frozen. Section 5 reports the results, and Section 6 what changed after the freeze and how the analysis was run. Section 7 discusses what the results mean, with the threats to validity, and Section 8 concludes.

2. Background and Related Work

2.1. What the First Bytes Say

A listener can decide a protocol from the first bytes only if the client sends first. For the protocols this paper serves directly, the client does.

- **TLS.** The client opens with a handshake record that carries a ClientHello [11]. The record's content type and the handshake message type are fixed. The record's version field is a legacy field that receivers must ignore, so only its major version is a usable key.
- **HTTP/2 with prior knowledge.** The client opens with a fixed connection preface of 24 bytes [10]. The other way to start HTTP/2 on cleartext, the `Upgrade` header, is deprecated.
- **HTTP/1.1.** The client opens with a request line: a method token, one space, a target and a version [13]. A server should ignore at least one empty line before it. The methods are those of HTTP semantics [14] and PATCH [15].

- **SSH.** Each side sends an identification string that starts with `SSH-` [16]. The standard does not say which side sends first, so a first-bytes listener can recognise only clients that send first.
- **MQTT.** The first packet of a client must be CONNECT. Its protocol name follows a Remaining Length field of variable size, so the name's offset depends on the length [17, 18].
- **PROXY.** A proxy may prepend a header that carries the original addresses, in a text form (v1) or a binary form (v2). The specification requires the receiver to be configured to expect it, and never to guess whether it is present [12].

Server-speaks-first protocols, such as SMTP, have the server send a greeting first [19]. A compliant client of such a protocol waits. Its connection delivers no byte from which to decide, so a first-bytes listener can serve it only after a timer.

The signatures overlap in ways a detector must handle. The binary PROXY header starts with an empty line, which an HTTP/1.1 parser would skip. The HTTP/2 preface parses as an HTTP/1 request line with the method `PRI`. `SSH-` is a legal prefix of an HTTP method token. An inline command of the Redis protocol, such as `GET key`, shares its first bytes with an HTTP request. A server that tells such classes apart needs a rule that does not depend on the order in which it tries them.

2.2. Systems That Detect a Protocol from Its First Bytes

We read the documentation and the source code of the systems below at the versions cited. Each statement here holds for that version.

Proxies. nginx's stream module reads the ClientHello before any handshake to route by its server name [1]. It peeks with `MSG_PEEK` only when it does not terminate TLS and its events are edge-triggered, on epoll with peer half-close reporting or on kqueue. Otherwise it reads the bytes and replays them to the upstream. HAProxy inspects content in its TCP mode by reading into its buffer and evaluating its rules again on each new chunk, for at most its inspection delay [2]. A connection that matches no rule passes to the default back end; the documentation discusses that delay with SMTP as its example. Envoy runs listener filters on the accepted socket [3]. Their shared buffer peeks with `MSG_PEEK`; of the filters we read, only the PROXY filter consumes bytes. caddy-l4 tries ordered routes of matchers on a recorded buffer that it rewinds after each matcher, on top of Caddy [4, 20]. sslh reads the client's bytes, runs its probes on everything read so far, and connects to a configured protocol when its timeout expires [5]. sshttp peeks once and chooses between SSH and HTTP or HTTPS [21]. Traefik splits TLS from other traffic by its first byte and reads and replays the bytes [22].

Libraries. Netty's port unification example reads a few bytes without consuming them, rewires the connection's pipeline, and hands the accumulated bytes to the next handler [6]. Jetty's detector asks each detecting connection

factory in turn and copies the unread bytes into a new buffer for the chosen one [7]. cmux records the bytes it reads and lets each matcher start again from the first byte [8]. hyper-util's automatic connection reads at most the HTTP/2 preface to choose between HTTP/1 and HTTP/2, and replays it [9]. Go's HTTP server checks the preface in user space when unencrypted HTTP/2 is enabled [23].

What they share. Most of these systems read the first bytes into user space and replay them; Envoy, and nginx on some paths, peek instead. None that we read documents a comparison of the two. A server-speaks-first protocol shares a detecting listener in two ways only. One is a timer, as HAProxy's default back end, sslh's timeout target and Envoy's default filter chain provide. The other is a listener that speaks first itself, as sshttp's SMTP mode does. Built-in detectors for SSH or MQTT appear in proxies (sslh and caddy-l4 for SSH, HAProxy for MQTT) and in none of the libraries above. Their documents do not say what happens when the first read is shorter than a signature. In the code, most of them wait for more bytes; sshttp decides on its single peek.

2.3. Kernel and Operating System Mechanisms

On Linux, a group of sockets can share a port with `SO_REUSEPORT`. A program attached to the group selects the listener for a new TCP connection at its SYN, before any application byte exists [24, 25]. A socket-lookup program selects a socket by addresses, ports and interface, and sees no payload [26]. `TCP_DEFER_ACCEPT` delays the accept until data arrives, on the same socket [27]. A socket map can redirect the data of an established socket, not the connection [28]. FreeBSD's accept filters hold a connection back from `accept` until a condition on its data holds [29]. On Windows, `AcceptEx` can return the first block of data with the accepted socket; with a receive buffer, it completes only after data arrives [30]. In the documentation and source code we read, none of these mechanisms routes a TCP connection to a listener by its first application bytes. They decide earlier, or they only delay.

2.4. Standards and Research

TCPMUX served many services on one TCP port by having the client name the service in its first line [31]. It is now historic [32]. ALPN negotiates the application protocol inside the TLS handshake, for several protocols behind one port [33]. For UDP, a receiver can demultiplex several protocols by the first byte of each datagram [34].

We found no peer-reviewed paper whose subject is a server that serves several protocols on one listener. Four lines of work touch the question. ALPACA shows that TLS does not bind a connection to the application protocol its client intended [35]. Work on probe-resistant proxies shows that how a server reacts to first bytes it does not recognise, including its timeouts, is a fingerprint [36, 37]. An analysis of STARTTLS finds defects where bytes read before a protocol switch are processed after it [38]. A first-bytes server hands bytes it has already read to the next protocol, and must do so correctly. Traffic classification infers an

application protocol from the content or the first packets of a flow, on monitors rather than on servers [39, 40, 41].

2.5. Gaps

Four gaps follow from the sources above. No system we read publishes a measured cost of its detection. None documents a comparison of peek with read and replay. Server-speaks-first protocols are served only through a timer, or by a listener that speaks first. What happens when the first read is shorter than a signature is mostly undocumented. This paper measures the first two for one server. It turns the last two into rules of that server, stated before any run and tested on a table of hard cases.

3. The Server

`oneport` is the paper's own minimal server, written in C++23 and built with CMake. This section describes it as the frozen design specifies it and as the code implements it. It holds exactly the features the hypotheses study, and nothing more.

3.1. Structure

One binary takes four flags: the mode (one-port, dedicated or stub), the detection mode (replay or peek), the dispatch mode (in-process or relay) and the backend (epoll, io_uring or IOCP). A process runs one backend. It runs one worker thread per server core; the primary cells use one core. The event loop of each worker derives from the minimal loop of an earlier study by the same author [42]. From it come the io_uring ring on raw system calls, created for a single issuer with deferred task work and enabled on the worker. So do the rules for memory that the kernel fills, under MemorySanitizer, the stop paths and the IOCP port setup. Sockets, timers, detection, handlers and counters are new.

A connection's state is a tagged union of handler states, and dispatch is a switch on the tag, with no virtual call. A dedicated listener sets the tag at accept; a one-port listener sets it when detection classifies the connection. From then on both modes run the same code. The mode flag is read once, by the listener setup. A structural test fails if any other source file reads it, so the handlers cannot differ between the modes.

3.2. Listeners

In one-port mode a process has one listening TCP socket. When several workers serve it, they share it: on epoll each worker's set holds it with `EPOLLEXCLUSIVE`, and on io_uring each worker's ring holds a multishot accept on it. On IOCP every listener keeps 64 `AcceptEx` requests outstanding, without a receive buffer, in both modes [30]. A group of `SO_REUSEPORT` sockets, one per worker, is a second layout that the binary also builds. On IOCP the server runs one worker and refuses more, so the secondary cell with two server cores

on IOCP was not run (Section 5.4). In dedicated mode a process opens one listening socket per protocol (HTTP/1.1, h2c, TLS, MQTT, SSH and SMTP) on consecutive ports. It has no detection, no detection timer and no detection budget. The backend, the workers, the handlers, the buffers and the PROXY setting are the same in both modes. That is the whole difference between them, and it is what the cost family measures.

3.3. Listening Descriptors

If the first bytes of the protocols on one transport are disjoint, the accepting host needs one listening descriptor per transport and accept model, not one per protocol. With A accept models and a set T of transports, that number is

$$L = A \cdot |T|. \tag{1}$$

Dedicated mode is the other layout, one listening socket per protocol. The cost family compares the two layouts. It does not ask the kernel how many listening sockets either layout holds. The server's own listening lines are not that count.

On 22 October 2026 the measured Release binaries were started with no client and no load. The kernel count on W was read with `netstat`, and the same addresses came from `GetExtendedTcpTable`. On L it was read with `ss`, over the management path to that host, still on 127.0.0.0/8. Table 1 gives the counts. No byte was sent, so this census does not measure classification. TLS was not on this path. The cost family did measure TLS exchanges, and Section 5.1 gives their power.

Table 1: Listening sockets the kernel reported for the measured binary. One-port mode is the shared listener unless the row names reuseport. Dedicated mode opens one socket per protocol. No client connected.

Host	Backend	Layout	Kernel count
W	IOCP	one-port, shared listener, one worker	1
W	IOCP	dedicated, one socket per protocol	6
L	epoll	one-port, shared listener, one worker	1
L	epoll	dedicated, one socket per protocol	6
L	epoll	one-port, SO_REUSEPORT, two workers	2
L	io_uring	one-port, shared listener, one worker	1
L	io_uring	dedicated, one socket per protocol	6

The shared listener is one accept model on one transport. Its kernel count is 1 on epoll, 1 on io_uring and 1 on IOCP. Dedicated mode's kernel count is 6 on epoll, 6 on io_uring and 6 on IOCP. With two workers in a `SO_REUSEPORT` group, ss reported 2 listening sockets on the one port. The process printed a single listening line for that port.

3.4. The Detection Table

Each matcher is a `constexpr` descriptor. Its function maps the bytes received so far to one of three answers: yes, no, or more needed. It also states

the fewest and the most bytes it needs to say yes, and the set of first bytes it can accept. The set is derived from the function, so the two cannot disagree. Table 2 gives the matchers as frozen. The methods of HTTP/1.1 are the eight of HTTP semantics [14] and PATCH [15].

Table 2: The detection table, as frozen. The PROXY parser is not part of the table; it runs before it, on a listener configured for PROXY only.

Matcher	Accepts	Decides at
TLS	first byte `0x16`; second byte `0x03`; third byte any; a record length of at most 16,384; sixth byte `0x01`, a ClientHello [11]	6 bytes
h2c	the preface of 24 bytes, exactly [10]	24 bytes, or the first byte that differs
HTTP/1.1	at most one leading empty line, then one of the nine methods, then exactly one space [13]	at most 10 bytes
SSH	`SSH-` [16]	4 bytes
MQTT	first byte `0x10`, a CONNECT; a Remaining Length of 1 to 4 bytes; then the protocol name `MQTT` with its length, and the level of MQTT 3.1.1 or of MQTT 5.0 [17, 18]	9 to 12 bytes
PROXY v1, v2	v2 needs 16 bytes with the first 13 matching; v1 needs 8 with the first 5 equal to `PROXY`; a prefix that cannot match closes the connection; a v2 header longer than 536 bytes is rejected [12]	then the whole header

From the table the compiler builds an array of all byte values that maps a first byte to its candidate matchers. Within a bucket, the matcher with the smaller need runs first, for speed only. Checks at compile time run every matcher over a corpus of samples built from the specifications' grammars. Each sample is accepted by its own matcher. No matcher accepts a prefix of another protocol's sample. Running each bucket in reverse order changes no decision on the corpus. The first-byte sets equal those of the frozen table. A build that broke any of these would not compile, so on the corpus the classes are prefix-disjoint and the order of the table cannot change a decision. The test suite runs the same corpus again at run time, split at every position.

Classification keeps no state between calls. The bytes stay where the de-

tection mode keeps them (Section 3.5), and the table runs on all of them each time more arrive. So a signature fed in any split gets the decision it gets in one write. An empty candidate set after the first byte rejects the connection at once. Two budgets are compile-time constants. An undecided connection may hold at most $B_{\text{dec}} = 24$ bytes after any PROXY header. A ClientHello reassembled in pass-through may hold at most $B_{\text{CH}} = 16{,}384$ bytes. A connection that exceeds a budget while it is undecided is rejected. On a listener configured for PROXY, the header is parsed first and consumed exactly, so the table sees the first application byte. That order removes the overlap of the binary PROXY header with HTTP/1.1's leading empty line.

3.5. Detection Modes

Replay. The first read goes into the buffer the handler would read into in dedicated mode. The matchers run on it, and the handler then parses the same buffer from its start. No byte is copied for the replay.

Peek. The bytes are copied with `MSG_PEEK` into a scratch buffer of the worker and stay in the kernel. The handler later reads them as in dedicated mode. An undecided peek sets `SO_RCVLOWAT` to the fewest bytes in all with which some candidate could still say yes. The socket then reports readiness again only when that many bytes are queued [24]. Before the handler reads, one more call sets the mark back to one byte.

The backends realise the two modes differently.

- **epoll.** Every connection is registered edge-triggered with `EPOLLRDHUP`, in both modes [43]. A readiness event is followed by a synchronous receive (replay) or peek. `EPOLLRDHUP` reports a peer that shut down writing, which a peek alone cannot show while bytes are queued.

- **io_uring.** In replay, a read on the socket is posted with a buffer that the kernel selects from a ring of provided buffers when data arrives [44]. A pending read therefore holds no buffer of its own. The opcode is `IORING_OP_READ`, a reading the revision log records. In peek, the worker posts a poll for readable data or a half-close, then peeks synchronously. A test pins that a poll armed again after an undecided peek completes only at the low-water mark.

- **IOCP.** A zero-byte `WSARecv` serves as the readiness signal, followed by a synchronous receive or peek on the non-blocking socket [45]. Windows has no `SO_RCVLOWAT`, so nothing can wait in the kernel for more bytes. An undecided peek therefore switches the connection to replay: the queued bytes are read into the handler's buffer at once and matching continues there. The counters record each switch.

Each backend has one default detection mode, used by every cost and robustness cell. The proposed default is replay on all three backends. Rule E (Section 4.7) may change it per backend after the pilot.

Memory of a pending connection. By design, a pending connection holds its state, which is fixed in size, and no data buffer while no byte has arrived, on every backend. With replay it holds the handler's buffer from the first byte on; with peek, nothing in user space. In pass-through it holds the partial ClientHello, at most B_{CH}. The test suite audits these bounds. IOCP has a second receive form that posts the handler's buffer with the first `WSARecv`. It holds a buffer while no byte has arrived, so it breaks the bound; it is built only for one secondary cell, reported with that violation.

3.6. Timers and the Decision Deadline

Three timers bound detection. The silence timer T_{fb}, the decision deadline T_{dec} and the PROXY header deadline T_{hdr} are each 3 s, a design choice. The PROXY specification asks for at least that long [12], and it is caddy-l4's default matching timeout [4]. T_{fb} and T_{dec} start together, at accept, or on a PROXY listener when the header is complete. A listener may name one fallback protocol. The frozen rules are these.

label=(a) A connection that has delivered no application byte when T_{fb} expires goes to the fallback handler. It goes in the first loop pass that handles the expiry, never earlier, unless the check of (b) finds a byte.

lbbel=(b) Every loop pass handles all of its completions and readiness events before any expiry. Before every fallback dispatch the server checks for a byte without waiting. Where no receive was posted with a buffer it peeks one byte with `MSG_PEEK`; where one was, it reaps completions without waiting. If it finds a byte, the bytes alone decide the connection.

lcbel=(c) After a fallback dispatch, the byte transcript in both directions equals that of the same client against the fallback's dedicated port. Only the timing differs.

ldbel=(d) A connection with at least one byte that no matcher has decided at T_{dec} is closed as undecided. It never goes to the fallback.

lebel=(e) A connection whose bytes rule out every matcher is closed at once as rejected.

lfbel=(f) A connection that is silent at T_{dec}, on a listener without a fallback, is closed as silent.

lgbel=(g) A client whose first byte is delivered after T_{fb} goes to the fallback. That is the defined price of the boundary, not an error.

lhbel=(h) A connection whose peer shuts down writing before a decision is closed at once: undecided if it delivered a byte, silent if not.

In relay dispatch a TLS connection is classified at its sixth byte, but its route needs the whole ClientHello. T_{dec} stays armed until the route is chosen, so it bounds that wait too.

The deadline queue. Every pending connection of a listener waits under the same durations, so its deadlines arrive in accept order. One intrusive first-in,

first-out list per listener and timer therefore keeps them sorted, with constant-time arming, cancelling and expiry. The loop bounds each wait by the earliest head deadline: `epoll_pwait2` with a time specification, the io_uring wait with its timeout, and `GetQueuedCompletionStatusEx` in whole milliseconds. After every wait the loop reads the clock again. If no deadline has passed it waits again, so no timed event is handled early. For each timed event the server records its deadline, when its wait returned and which pass handled it.

3.7. Dispatch

In-process. The classified connection stays on its worker, and its handler takes over. TLS is terminated in the server.

Relay. Relay dispatch exists on Linux only. The front opens a loopback connection to the back-end port of the connection's class, writes any bytes replay has read, and copies both ways. A side's end is passed on as a half-close, and the connection closes when both directions have ended; a reset on one side closes the other by reset. For TLS the front never terminates. Its own parser reassembles the ClientHello across records, up to B_{CH}, and routes by its server name and ALPN list (pass-through). The relay copies through user-space buffers of 4,096 bytes per direction, or with `splice`; rule E chooses per backend.

Stub mode. The back end of the comparisons with the proxies is the server in stub mode: one port per class, and the least work that completes an exchange. HTTP/1.1 gets the handler's response. On the TLS port the stub reads one TLS record, writes a body of 13 bytes and closes; it never makes a handshake.

3.8. Handlers and TLS

Every handler is the same code in both modes, and each is minimal. HTTP/1.1 is the server's own parser; it answers status `200` with a body of 13 bytes. A request line that breaks the detector's grammar is closed without a response in both modes, so the modes agree on invalid input. HTTP/2 runs through nghttp2 version 1.70.0 [46]. MQTT answers CONNECT with CONNACK and PINGREQ with PINGRESP, and closes on DISCONNECT. SSH sends its identification line `SSH-2.0-oneport` on entry, reads the client's line and closes. SMTP, reached only as a fallback or on its dedicated port, answers `220`, then `250` to EHLO, `221` to QUIT and `500` to anything else [19].

TLS runs through OpenSSL from its 3.5 long-term support series, pinned at release 3.5.9 and built from source on both hosts [47]. The settings are the same in the server, the back end and the generator. They are TLS 1.3 only, the cipher suite `TLS_AES_128_GCM_SHA256`, the signature scheme `ecdsa_secp256r1_sha256` and the group X25519 only. There are no session tickets (`SSL_CTX_set_num_tickets(ctx, 0)`), no session cache and no early data. The cost cells offer ALPN `http/1.1`. The loop owns the socket on every backend and gives OpenSSL its bytes through a BIO of the worker's own. io_uring and IOCP complete operations that the library cannot issue itself.

3.9. Counters

Every build counts, per worker, in plain integers without atomics, in both modes. It counts operations by kind: accept, receive, peek, zero-byte receive, send, `setsockopt` and the check of rule (b). It also counts `io_uring_enter` calls, submissions by opcode and `GetQueuedCompletionStatusEx` calls. Further counters hold the switches from peek to replay and the payload bytes across the boundary between user and kernel space, by direction. Others hold the bytes copied in user space, the wakeups per connection during detection, the outcomes per class and the timed events above. The counters run in every arm, so they cost the same on both sides of each contrast.

3.10. Generators and the Holder

Three first-party programs drive and observe the server. `opgen` generates load: scripted exchanges per protocol, a new connection per exchange or kept-alive connections, closed and open loop, and the time to the first response byte. It binds each source address with `IP_BIND_ADDRESS_NO_PORT`, and each window takes the next unused block of 16 addresses of 127.0.0.0/8, never reused within 60 s. Windows has no such option, so on W `opgen` sets `SO_REUSE_UNICASTPORT` before each bind, a reading the revision log records. Two facts go with it, both from development runs on W. A port is named right after the bind with or without the option, so the port is not shown to be chosen at connect. And the generator's churn rate did not change without the option. `opcase` runs each hard case, and the openings of the footprint test, as a script of writes and gaps, one write per chunk with `TCP_NODELAY`. It records each write's time on the server's clock. `ophold` accepts connections and holds them without reading or polling them. It measures the kernel's memory for a bare held socket.

4. Method

Every rule in this section is taken from the pre-specification, which was frozen on 2 October 2026 before any code of the server existed. Its text was never edited after the freeze. Readings of it made during engineering, values set by its rules, and the decisions of the author went into its dated revision log. That log is part of the frozen file. Where this section states such a reading, it says so.

4.1. Families and Hypotheses

The hypotheses form three families, and no contrast crosses families. No proxy is compared with the server's in-process mode, and no result of the mechanism family supports a claim of cost. Table 3 lists the hypotheses.

Table 3: The hypotheses, as frozen. A ratio's numerator is the first arm named. "Holm" marks a family tested with Holm's step-down procedure at family-wise $\alpha = 0.025$, one-sided.

Id	Claim	Bound	Cells
C1	Churn throughput (connections per second, WL1), one-port / dedicated, is equivalent	within $[0.98, 1.02]$	C1 to C3:
C2	Keep-alive throughput (requests per second, WL3) is equivalent	within $[0.98, 1.02]$	at most 36,
C3	Median time to first response byte at a fixed load (WL2) is equivalent	within $[0.98, 1.02]$	Holm
B1	The outcome and transcript of every hard case equal the frozen table	none deviates	deterministic
B2	Timed events are never early and are handled in the first pass after their deadline; pending memory stays within its bounds	none deviates	deterministic
B3	Counted footprint per pending connection, competitor / server, exceeds the margin	median $Q > 1.10$	18, Holm
M1	The default detection mode is cheaper: churn throughput, default / other	median > 1.00	6
M2	In-process dispatch costs less CPU than relay: CPU per connection (WL6), in-process / relay	median < 1.00	4
M3	The server's relay is faster than each proxy: churn throughput, server / proxy	median > 1.00	10

Cost family. The server in one-port mode is compared with itself in dedicated mode: the same binary, in-process dispatch, the default detection mode and one protocol per cell. The cells cross C1, C2 and C3 with HTTP/1.1, h2c, TLS with HTTP/1.1 and MQTT. They run on epoll and io_uring on L and on IOCP on W, at most 36 in all. The confirmatory family is the subset the A/A pilot resolves (Section 4.6.1). Its size is $m_C = 32$. Stated before the run: the TLS handshake dominates a TLS connection under churn. So the TLS cells have little power to show the cost of a check of 6 bytes, and they are not read as evidence about it.

Robustness family. B1 and B2 are deterministic and outside Holm. B1 runs every hard case of the frozen table on every backend, in both detection modes, with every dispatch mode that applies, 16 times. The replicates exercise the timing of arrival. One deviation fails B1. A connection classified other than as its script's protocol, in any measured window of any family, also fails B1. B2 holds five bounds.

label=(a) No fallback, close or deadline event happens before its deadline on the server's clock.

lbbel=(b) Each deadline is handled in the first loop pass whose wait returned at or after it.

lcbel=(c) Each connection is classified in the pass that received its deciding byte.

ldbel=(d) No pending connection holds more user-space payload than its mode's bound, and one that has received no byte holds no data buffer.

lebel=(e) A half-close before a decision closes the connection in the pass that observes it.

The lateness of timed events and the decision latency are reported per backend as distributions, and not tested.

B3 compares the counted footprint per pending connection, W (Section 4.2.1), of each competitor as configured with the server's. The author fixed its margin before any window of B3 ran: a cell counts only if $W_{\mathrm{comp}}/W_{\mathrm{srv}}$ is at least 1.10. The session statistic is $Q = W_{\mathrm{comp}}/W_{\mathrm{srv}}$, each arm's W the mean of its two windows. A session whose W_{srv} is zero or less takes $Q = 0$, which counts against. The server runs in relay mode against the proxies and in-process against the libraries, in its default detection mode, with every detection timer at 60 s. The 18 Holm cells cover two cases, each on epoll and io_uring. The silent case runs against nginx, HAProxy, Envoy, sslh-ev and hyper-util (10 cells). The partial-ClientHello case runs against the first four (8 cells), since hyper-util serves no TLS. Netty, Jetty, caddy-l4 and cmux give 16 more cells, reported against the same bound and deciding nothing: they measure a runtime's collector as much as a detector.

Mechanism family. M1 compares the two detection modes for HTTP/1.1 and h2c churn on all three backends, in the direction rule E fixes (Section 4.7). On IOCP the peek mode includes the switch to replay, so M1 there measures the limits of the API as much as the idea. M2 compares the CPU time of the front and the back end together, per connection, for HTTP/1.1 and TLS on epoll and io_uring. The relay arm passes TLS through to a back end that terminates it, so both arms make one handshake per connection. M3 compares the server's relay with each of the five proxies, each with one front core, in front of the stub. It covers TLS routed by its server name and plaintext HTTP/1.1. The server runs on epoll, the accept model all five use. Each system holds exactly two routes: TLS by server name to one stub port, and everything else to the HTTP/1.1 stub port. What each examines to choose differs: nginx tells TLS from not TLS, while the server runs its whole table.

4.2. Workloads and Metrics

WL1, churn in closed loop. $C = 64$ connection slots per server core. Each slot repeats connect, one exchange and close (Table 4). The metric is connections per second: the exchanges completed without error per second of the measured window.

Table 4: The exchanges of WL1.

Protocol	Exchange	Closes first
HTTP/1.1	`GET` with `Connection: close`; read the response	server
h2c	preface, SETTINGS, one HEADERS frame with END_STREAM; read to END_STREAM; GOAWAY	client
TLS (HTTP/1.1)	full handshake, then the HTTP/1.1 exchange	server
MQTT	CONNECT; read CONNACK; DISCONNECT	client
SSH (secondary)	send the identification line; read the server's	server
TLS against the stub (M3)	one recorded ClientHello, the same bytes on every connection; read the stub's 13 bytes to the end	server

WL2, churn in open loop. Exchanges fall due at a rate λ, 0.5 times a median taken over the A/A pilot's sessions. Each session contributes its mean connections per second in the C1 pilot cell with the same protocol and backend. The factor is a design choice, below saturation. The time to first response byte ends at the first byte the server sends on the connection, as the generator receives it. In open loop it runs from the exchange's due time, so a stall of the generator cannot hide. The metric is its median. A window in which fewer than 99% of the exchanges due completed is invalid.

WL3, keep-alive. C connections are established before the window, each with one request in flight. The request is an HTTP/1.1 `GET`, one h2 stream at a time, MQTT PINGREQ after one CONNECT, or an HTTP/1.1 `GET` over one TLS connection. The metric is requests per second. A connection's setup is not a request.

WL4 to WL6. WL4 is secondary: the server's CPU time over the window per exchange, and its resident memory at the end and at its peak. WL5 is secondary: the counters of Section 3.9 per connection. WL6 is M2's metric: the CPU time of the front and the back end together, per connection completed, at an open-loop rate. The rate of each M2 cell is 0.5 times the smaller of its two arms' median closed-loop connections per second. Those medians come from 6 development sessions on the frozen binary after the pilot. The relay arm's back end runs in a `SO_REUSEPORT` group of two workers, a design choice the revision log records with its reason.

WL8, hard cases. For each case, the outcome, the transcript and the decision time against `opcase`'s write times. The table holds 25 cases. They cover whole, split and dripped signatures of every protocol, and slow drips still incomplete at T_{dec}. They cover silent clients with and without a fallback, and a first byte just after or just before T_{fb}. PROXY headers come whole, split, missing, incomplete, too long and followed by each protocol. The rest are bytes that rule

out every matcher, half-closes and resets during detection, and a Redis inline command, answered with `400`.

4.2.1. WL7, the Footprint per Pending Connection

A connection is pending while the client's bytes are incomplete, whatever the system has decided from them so far. Every B3 window lasts 30 s from the first connect, in phases that are design choices. Before $t = 0$ the processes start fresh, pass the probe and take a baseline sample. From $t = 0$ to at most 10 s `opcase` opens 10,000 connections in batches of 25, one every 25 ms. A batch is half of sslh's hard-coded listen backlog of 50, the smallest among the systems. Each connection then sends nothing (the silent case), or the record header and the first half of the recorded ClientHello of 206 bytes (the partial-ClientHello case). After settling to $t = 20$ s, samples follow at $t = 20$ s and $t = 25$ s, and the second is the value used. Every connection is then closed by reset, so no socket enters TIME-WAIT. Each sample reads three parts, per pending connection. U is the growth over the baseline of the summed resident memory of the system's processes; the stub behind a relay holds no pending connection and is left out. K_q is the mean receive-queue allocation (the `r` field of `ss -tm`) over the system's accepted sockets. It sums the true size of the buffers queued for reading, which holds their payload and their headers. K_s is the growth over the baseline of the kernel's slab memory, less the growth of the socket-buffer caches. It is host-wide and holds both loopback ends; the client ends are `opcase`'s, the same for every system.

A queued socket buffer's header is itself a slab object, so the slab total holds it while K_q charges it too. In the Linux source at tag v7.2 [25], those headers come from three caches, whose growth is subtracted from the slab's, so each queued buffer is counted once. On L one of the three is merged into a shared cache with `pool_workqueue` and `sgpool-16`, and the whole shared cache's growth is subtracted. Two residuals remain, both with a system that leaves bytes queued. A header too large for its cache is counted twice, which raises that system's W. Memory of the caches that no receive queue is charged for is removed, which lowers it. In the silent case neither applies.

The holder `ophold` measures K_{BASE}: it holds 10,000 silent connections in the same layout, and K_{BASE} is the median of its K_s over 16 windows, corrected the same way. A system's footprint is $T = U + K_q + (K_s - K_{\mathrm{BASE}})$, and its counted footprint is $W = T + K_{\mathrm{BASE}} = U + K_q + K_s$. W is the footprint of each system as configured, not of its detection design alone, and it is not the whole system's. Outside it are the reserved but unused part of a socket's receive memory, and kernel memory outside the slab that no receive queue is charged for. The second residual and the co-tenants' growth of a merged cache are outside it too, and the first residual can count bytes twice. K_{BASE} cancels from every statistic of B3. It is reported as the share of each footprint that every system pays alike.

Every process raises its limit of open files to the hard limit. Connection limits are unset or at least 10,000, and listen backlogs are 10,000 wherever a

system has a setting. Runtimes with a collector collect before each reading, the baseline included.

4.3. Competitors and Their Configurations

Table 5 lists the competitors and their pins. Every configuration file is in the paper's repository, every line with its reason. The rule is each project's documented best configuration for the compared feature, with logging of individual connections off.

Table 5: Competitors, pins and families.

System	Pin	Families
nginx, stream module [1]	1.30.5, built from the release	B3, M3
HAProxy [2]	3.4.6	B3, M3
Envoy [3]	1.39.2, the release binary	B3, M3
caddy-l4 [4, 20]	v0.1.2 on Caddy v2.11.4, built with xcaddy 0.4.7 and Go 1.27.1	B3 (descriptive), M3
sslh, the `sslh-ev` binary [5]	2.3.1	B3, M3
Netty [6]	4.2.18.Final, on the Temurin JDK 25.0.4.1+1	B3 (descriptive)
Jetty [7]	12.1.13, on the same JDK	B3 (descriptive)
cmux [8]	v0.1.5 on Go 1.27.1	B3 (descriptive)
hyper-util [9]	0.1.21, with Rust 1.98.1	B3

Each proxy has three configurations. The *cases* configuration holds every route its features cover, the matched timers and the fallback where it has one. The *M3* configuration holds exactly M3's two routes, with timers matched to the server's 3 s and one front core. The *B3* configuration adds WL7's limits, the 60 s timer and the buffer settings below, and gives the competitor the server's core count. Each library harness has the *cases* and the *B3* configurations. caddy-l4 is pinned by its release and its commit, `42db5690dea199f930a6f08005fe2e4aab10dcc9`.

Proxies. nginx runs one worker on the front core, with the event method it chooses, which on Linux takes its peek path, and routes with `ssl_preread`. HAProxy runs in TCP mode with one thread, accepts a ClientHello by its hello type and plain HTTP/1.1 at once, and routes by server name. Its client, server and connect timeouts stay unset, its default. HAProxy gets `option splice-auto` only where the server's relay in the same cell splices. Envoy follows its benchmarking advice: one worker thread, circuit breaking disabled, filter chains only for the compared features, and its default statistics on. caddy-l4 runs with one Go processor. sslh runs its `sslh-ev` binary with per-connection logging off.

Libraries. The Netty harness follows Netty's port unification example on the native epoll transport. The Jetty harness uses its detecting connection factory with TLS and PROXY, then HTTP/1.1 with the h2c upgrade. Both

JVMs run the G1 collector with the JDK guide's default heap bounds. The cmux harness uses cmux's matchers for HTTP/2, HTTP/1, TLS and `SSH-`, with a match-all last for the fallback. The hyper-util harness serves HTTP/1.1 and h2c on a tokio runtime with one worker thread and has no detection timer.

Buffers in B3. A default that sizes what a pending connection holds is changed in two cases only. Either the project documents a value for deployments that face many untrusted connections, or it documents that the buffer grows on demand, so a smaller start refuses nothing. So Envoy's per-connection buffer limit is 32,768 bytes on the listener and the cluster, as its edge guidance asks of TCP proxies. Its TLS inspector starts with 256 bytes, the smallest value it accepts. HAProxy sets small buffers in its back ends, which a pending connection does not use. Every other default is kept. The server's relay copies through 4,096 bytes per direction. Beside it, nginx's `proxy_buffer_size` defaults to 16 KiB, HAProxy's `tune.bufsize` to 16,384 bytes, and Envoy's per-connection buffer limit to 1 MiB; caddy-l4 and sslh fix theirs in code.

4.4. Hosts and Procedures

Hosts. L is a Linux host with an AMD Ryzen 7 5800H processor: 16 logical CPUs, with 2 hardware threads per core. It runs Arch Linux with kernel 7.2.6-arch2-1, clang 22.1.8 and Go 1.27.1. W is a Windows host with an AMD Ryzen 5 3600 processor, 6 cores and 12 logical CPUs, running Windows 11 Pro N (build 10.0.26200) and MSVC 19.51.36246. Every run uses the loopback interface. The pre-specification cites Linux source lines at tag v7.2; the revision log checked each cited line against the kernel the runs use and found none changed in content. L and W are two machines. No result in this paper pools them.

Placement. The hardware sibling of every used core stays idle, and the server, its back end and the generators never share a core. On L, CPUs 0 and 1 run the system and the driver; in in-process cells the server runs on CPU 14 and `opgen` on CPUs 2 to 13. In hand-off cells the front runs on CPU 14 and the back end on CPUs 10 and 12. On W the server runs on one logical CPU of the last physical core and `opgen` on the cores between core zero and the server's. The placement follows the earlier study of the same author [48].

L. Every run on L is a lab job under a lock. The job turns boost off, sets the performance governor, holds each CPU's minimum frequency at its maximum, sets transparent huge pages to `madvise` and exempts loopback traffic from connection tracking. It restores every setting when it ends, and each session's fingerprint records them.

W. For each session the harness switches to a power plan of its own. It is a copy of Windows' High performance plan with boost off, the processor state at its maximum and no core parking. The timer resolution stays at Windows' default. A quiet check before each job refuses a busy host. The frozen procedure also names a frequency counter, tested against a known load; if none passes, W's windows are validated without the frequency rule. The author approved W's procedure before the code freeze, and Section 6.5 gives what followed.

4.5. Sessions and Validity

A session is four windows of a cell's two arms in mirrored order, $XYYX$; which arm is X is drawn per session. Each window starts fresh processes, checks one exchange of each protocol the cell uses (the probe), then runs a warm-up of 1 s and a measured window of 5 s. The design follows the earlier study [48]. B3 windows have the layout of WL7. An arm's value in a session is the mean of its two windows. The session's statistic is the ratio of the two arms' values, in the direction each hypothesis states. Sessions are separate process lifetimes and are treated as independent. An analysis clustered by lab job is reported beside every family and decides nothing.

A window is invalid, and is listed with its reason, if any of these holds:

- the server, the back end or the probe failed, no exchange completed, or any connect failed;
- on L, the share of failed exchanges exceeded 0.1%, or the generator's CPUs were more than 90% busy in a closed-loop cell;
- on L, the mean CPU frequency drifted more than 2% from the session's value;
- on W, a rule of W's procedure that can be computed there failed;
- any connection was classified other than as its script's protocol (also a failure of B1);
- in a hand-off cell, a back-end core was more than 90% busy;
- in open loop, fewer than 99% of the exchanges due completed;
- in the cost family, M1, M2 and B3, the kernel's listen overflow or drop counter changed during the window;
- in a B3 or `ophold` window, W moved between the two samples by more than its settling bound;
- in such a window, the host's count of TIME-WAIT sockets at either sample differed from the baseline's, or fewer than 10,000 of the system's accepted sockets were established;
- in such a window, a socket-buffer cache could not be found or read.

The settling bound is the larger of 2% of the second sample's W and 8 bytes per pending connection. In M3 the proxies keep their own backlogs, so their listen overflows are reported beside each cell instead. TIME-WAIT sockets are slab objects that last 60 s, so the first B3 or `ophold` window of a lab job starts at least that long after any other window ended.

A session with an invalid window is discarded and run again at the end of its family's order, at most $\lceil R/4 \rceil$ times per cell. A cell with fewer than R valid sessions cannot pass and enters Holm's procedure with $p = 1$. Beside every decision the paper reports the cell's invalid windows per arm, with their reasons.

4.6. Statistics

Every tested cell is decided by two computations, both fixed before the run. A cell passes only if it passes under both; if they disagree, both are reported and the cell does not pass. The cell's statistic is the median of its R session values.

- **Clustered BCa interval.** 10,000 resamples of the sessions with replacement, the session as the cluster [49, 50]. The bias correction counts ties as half, and the acceleration comes from the jackknife over sessions. The share of resamples beyond a bound is clipped to $[1/(2B),\ 1-1/(2B)]$, with $B = 10{,}000$. The one-sided p-value against a bound is the level α at which the bound of the two-sided $1-2\alpha$ interval equals it.
- **Exact sign test.** X counts the sessions whose value lies strictly on the hypothesis side of the bound; a tie, or a session without a value, counts against. At the null's boundary $X \sim \text{Binomial}(R, 1/2)$, and $p = P(X \geq x)$ is computed exactly.
- **Holm's step-down procedure** [51] per family at family-wise $\alpha = 0.025$, one-sided, run once for each computation. A cell that cannot be tested enters with $p = 1$.

Equivalence in the cost family. The author fixed the margin before any code existed. Equivalence for C1, C2 and C3, on both hosts, is a ratio between 0.98 and 1.02. An interval that excludes one is a bounded difference. This paper does not call that difference absent. Each cell has two one-sided nulls: the median ratio is at most 0.98, and it is at least 1.02. Each is tested at α by both computations, and a computation's p-value for the cell is the larger of its two (two one-sided tests [52]). Before Holm's adjustment, the BCa form passes exactly when the two-sided 95% interval lies inside the margin. Each cost cell shows that interval and the interval at its Holm level, $\alpha_j = \alpha/(m_C - j + 1)$, where j is its rank. A noisy cell, or one with too few valid sessions, fails: a difference that is merely not significant never counts as equivalence. In the cost family the sign test is the binding computation, since at Holm's first threshold it allows fewer sessions beyond a bound than the BCa needs.

Superiority. Each cell of B3 and of the mechanism family has one null, on the far side of its bound from the hypothesis. For B3 the null is that the median Q is at most 1.10. For M1 and M3 it is that the median ratio is at most 1.00, and for M2 that it is at least 1.00. The sign test counts the sessions strictly on the hypothesis side. The BCa p-value is the level at which the lower end of the interval equals the bound (the upper end for M2).

Replicates. An exact paired test must be able to pass the family's first Holm threshold, $2^{-R} < \alpha/m$. That needs at least 11 sessions in the cost family, 10 for B3 and 10 for the mechanism family. B3 and the mechanism family run $R_B = R_M = 16$, the reference count of the earlier study [48], a design choice. No pilot can size a superiority family, since nothing in the design sets its effect size. At that R, one session on the wrong side of the bound still clears every Holm threshold of both families. There is no sequential stopping.

4.6.1. Sizing R_C from an A/A Pilot

The pilot sizes R for the cost family; it sets no margin. Every cost cell runs 16 sessions with both arms in dedicated mode. The arms are two starts of the same binary and flags, the second on a fixed port offset. So every window transition changes port, as in the A/B design. The pilot holds no one-port data. For each cell, with $y_i = \log r_i$ the log of session i's ratio, the values are centred, $e_i = y_i - \text{median}(y)$, so power is computed at a true ratio of one. With s their standard deviation, the bandwidth is Silverman's rule of thumb [53] divided by s, $b = 0.9 \cdot \min\bigl(1, \text{IQR}(e)/(1.34\,s)\bigr) \cdot P^{-1/5}$. The spread used is $s_U = s\sqrt{(P-1)/q}$, the one-sided 80% upper confidence bound under normality, with q the 0.20 quantile of the chi-square distribution with $P-1$ degrees of freedom. One simulated run draws R values e_i/s with replacement, adds to each a normal draw with standard deviation b, multiplies by $s_U/\sqrt{1+b^2}$ and takes exponentials. The confirmatory code analyses each run, and the run passes if both p-values at the margin are at most $\alpha/36$. The power at R is the share of 1,000 runs that pass. The candidates for R are 11, 15, 18, 22, 25, 28 and 31, where the sign test's allowed count steps. A cell is resolved if it has 16 valid pilot sessions and power of at least 0.80 at $R = 31$. R_C is the smallest candidate at which every resolved cell reaches 0.80. The pilot entry records $R_C = 28$ and $m_C = 32$. A cell not resolved leaves the confirmatory family, still runs at R_C, and is reported as not resolved at $R \le 32$. For each protocol, backend and host whose C1, C2 and C3 cells are all resolved, the product of their three powers is reported, not targeted.

Orders and seeds. All sessions of one family's cells on one host run in one shuffled order, so that drift spreads over the cells. The 14 seeds of the orders, of each family's resamples and of the pilot were drawn once on L, in one revision-log entry before the code freeze. Every integer that the lab journal or the engineering notes named was excluded, so no seed of a development run could recur. The notes, as they stood at that entry's commit, are in the paper's repository. The journal also covers the author's other work and is not published.

4.7. Order of Freezes and Runs

Each step starts only after the one before it is committed.

1. The pre-specification is frozen before any code of the server exists.
2. Engineering. Before it ends, one revision-log entry fixes every seed, and the analysis code that sizes R_C is committed with its tests, pinning numpy 2.5.0 on Python 3.14. Section 6.6 names the commits of that code.
3. The code freeze: one commit of the server, the generators, the holder, the harnesses, the pins and the tests, with green sanitizer records and a passing test suite.
4. On the frozen binary, in dedicated mode only: the A/A pilot on each host, with its timer and split parts.
5. The simulation that sizes R_C, on the pilot's data alone.

6. The pilot entry: one revision-log entry made from the pilot's and the simulation's outputs alone, committed before any one-port window of the frozen code. Nothing in it is chosen.
7. After it: rule E's choices, M2's rates, the footprint feasibility windows, K_{BASE}, and every confirmatory, secondary and hard-case run, each family in its own order.

The pilot's parts. The timer part runs 128 single connections per backend whose PROXY header stays incomplete, and records how late the loop handles T_{hdr}. Per host, G is the smallest whole number of milliseconds above the largest lateness. It sets the guard band of the hard case whose first byte arrives just after or just before T_{fb}, which runs only where G is below T_{fb}. The split part sends an HTTP/1.1 request split after its first byte, with gaps of 5, 10, 20, 50 and 100 ms, 16 times each. The smallest gap at which every replicate on every backend shows the split read becomes the gap of the split hard cases. The pilot entry set $G = 4$ ms on L and $G = 17$ ms on W, so that case ran on both hosts, and a split gap of 5 ms.

Rule E. Three choices per backend are fixed by a rule: the default detection mode, the receive form on IOCP, and the relay copy. Both options of each are built into the frozen binary as flag values. The proposed defaults are replay, the zero-byte receive and user-space buffers. An option replaces its default only if every one of at least 6 development sessions of HTTP/1.1 churn favours it and its operations per connection are not higher. The receive form that posts a buffer at once breaks B2's memory bound, so rule E cannot choose it, and the zero-byte form stands. The other two choices are made after the pilot entry, before any confirmatory window. No option replaced its default: the detection mode is replay on all three backends and the relay copies through user-space buffers. So HAProxy gets no `option splice-auto` in any cell.

Development data. Engineering ran development windows. They are never results. The revision log and the unpublished lab journal hold them, and this paper cites them only as reasons for decisions and readings. Before the code freeze they could time dedicated mode against anything, and one-port mode against a competitor or against another one-port option. No window, on any code, timed one-port mode against dedicated mode before the pilot entry. A dedicated window and a one-port window against the same competitor, at the same protocol, backend and host, would still give an indirect ratio of the two modes. The revision log lists every such pair when it runs. It lists none, and no rule uses one.

A change once the pilot has started. Before the simulation starts, a change forced by a named failure needs new records and a new pilot, at most once. The named failures are a failing test, a sanitizer record that is not green, the gate refusing a build, and a crash of a first-party program. Any other change keeps the pilot and gets new records. Every confirmatory window of the old build is archived, the runs start again on the new build, and the change is recorded as a deviation. No one-port window of either build runs before the pilot entry.

4.8. Sanitizer Gate and Declared Gaps

A number is citable only if every measured build compiled first-party inputs that a green sanitizer record covers. On L the records use clang 22.1.8 with AddressSanitizer and UndefinedBehaviorSanitizer together, ThreadSanitizer, and MemorySanitizer with an instrumented C++ standard library. On W they use MSVC 19.51.36246 with AddressSanitizer, one compiler because the gate matches the compiler. Each record of the server covers the server, `opgen`, `opcase`, the test suite and, on L, `ophold`, by a hash of their inputs. The suite runs the detection table at every split, the whole table of hard cases, every mode of the binary and every backend of its host. Under ThreadSanitizer two workers share one listener. The gate refuses a measured build unless green records on the same host cover every first-party target, with the same compiler, configuration, pins and fetched archives. The Go and Rust harnesses get AddressSanitizer and ThreadSanitizer records.

The repository's coverage file declares the gaps; these are the main ones. No sanitizer instruments OpenSSL's assembly, and MemorySanitizer cannot see the kernel's writes into io_uring's rings and buffers, so exactly the bytes a completion reports are unpoisoned. The competitors' binaries and runtimes are pinned, not instrumented, and the Java harnesses have no sanitizer. On Windows the rule does not require ThreadSanitizer or MemorySanitizer.

4.9. Secondary Analyses and Cells Not Run

Secondary analyses decide nothing and are in no Holm family. Cells that need sessions run at $R = 16$, and each result is reported with its 95% interval where it has one. They are these.

- The CPU per exchange and resident memory of every cost cell, and the tail of the time to first byte.
- The cost cells that the pilot did not resolve.
- SSH in the cost family (9 cells). The server's identification line waits for the client's in one-port mode but is sent at accept in dedicated mode, so the message order differs by mode.
- A mixed-protocol cell per backend (3 cells): HTTP/1.1 churn beside a fixed background of 64 kept-alive TLS, 64 kept-alive MQTT and 64 silent connections.
- TLS with session resumption and TLS with ALPN `h2` (6 cells).
- Two server cores (3 cells), and a `SO_REUSEPORT` group against the shared listener (2 cells).
- The server's relay on io_uring against the proxies (10 cells).
- Two accept and receive forms on IOCP (2 cells). One is the posted form, which breaks B2's memory bound; the paper states that beside it.

- The time to first byte at a fixed load for M1 and M3 (16 cells).
- B3 with the server in its other detection mode (4 cells).
- Operations and copies per connection.
- The competitors' outcomes on the hard cases they cover, at matched timers and at their defaults, 3 replicates each. A system with no timer is observed for at most 60 s.
- The distributions of B2.

On L, one untimed window per cost cell checks the server's system-call counters against the kernel's count of the same calls' entry tracepoints, read with `perf stat`. `perf trace -s` stays beside it as a cross-check, and its shortfall is reported beside the counts. The proxies' system calls per connection come from `perf trace -s` over untimed windows of the same kind. An untimed window holds counts only, and is not a window of any cell.

Four kinds of cell are reported as not run, each with its reason. TLS with session resumption (3 cells) is not run. The frozen settings of every endpoint exclude session tickets and a session cache, and TLS 1.3 resumes only from a ticket. An M2 cell whose rate its rule cannot compute, because an arm has no valid development session, has no rate. It runs no window, enters Holm with $p = 1$, and keeps the family's size; it is reported as a difference not shown. Before the code freeze the author decided that M2's two TLS cells are such cells (Section 5.3). The churn of h2c and MQTT on W runs in the pilot only, and the secondary cell with two server cores on IOCP is not run (Section 6.1).

4.10. What Counts against the Claims

Cost. A cell that fails is reported with its interval as "equivalence not shown". It is called a measured cost only where its 95% interval lies wholly outside the margin. "No measurable cost" is claimed only as equivalence within the margin, and for load of one protocol. It is claimed only for a protocol, backend and host where C1, C2 and C3 all pass. The joint power and the intervals at Holm's level stand beside it.

Robustness. B1 and B2 are not narrowed: a failure in any run of the frozen code is reported as a failure, and the robustness claim is not made. B3 is narrowed to the competitors and cases where it passes. A pass is written as "the competitor held at least 1.10 times the server's counted footprint per pending connection". It always comes with the difference in bytes and its interval, and with WL7's exclusions and residuals named. "Loss" is used only where the 95% interval of Q lies wholly below one.

Mechanism. M1 and M2 are reported per backend as measured; a cell that fails shows only that a difference was not shown. M3 is narrowed to the proxies and protocols where it passes.

Narrowing order. The cost family leads the paper. If a family cannot be won cleanly, the claims narrow in a fixed order. M3 narrows first, to the proxies

and protocols the server beats. B3 follows, to the competitors and cases it beats. The cost family narrows last, per backend.

Analysis. The analysis computes every decision from the archived runs and writes the paper's numbers into a generated macro file; Section 6.3 records the one departure. Raw outputs are archived with their SHA-256 (Section 6.9). Nothing is run again to change a decision.

5. Results

This section reports each family's outputs as the frozen analysis wrote them, with every verdict in the analysis's own words. Every value comes from a generated macro file or a generated table. Section 6 gives what changed after the freeze and how the analysis was run. Table A.12 in Appendix Appendix A lists every invalid window of a family cell, per arm, with its reasons.

5.1. Cost

Table 6 gives every cell of the cost family. Of the 32 cells in Holm's procedure, 30 pass under both computations, and the two computations disagree on 0 cells. C1 passes in 10 of its 10 cells in Holm, C2 in 12 of 12, and C3 in 8 of 10.

Table 6: The cost family, one-port / dedicated. C1 and C2 are throughputs, so a ratio below one favours dedicated mode; C3 is the median time to first byte, so a ratio above one does. The intervals are the 95% BCa interval and the interval at the cell's Holm level. The verdict is the analysis's, cut before its first parenthesis. Where an interval excludes one, the bounded cost is that ratio.

Hyp.	Host, backend	Protocol	Valid	Median	95% interval	Holm interval	Verdict
C1	L, epoll	HTTP/1.1	28	0.9916	[0.9900, 0.9933]	[0.9887, 0.9946]	equivalent within [0.98, 1.02]
C1	L, epoll	h2c	28	0.9946	[0.9926, 0.9974]	[0.9915, 0.9982]	equivalent within [0.98, 1.02]
C1	L, epoll	TLS	28	0.9986	[0.9971, 0.9995]	[0.9966, 1.0000]	equivalent within [0.98, 1.02]
C1	L, epoll	MQTT	28	0.9921	[0.9910, 0.9932]	[0.9906, 0.9940]	equivalent within [0.98, 1.02]
C1	L, io_uring	HTTP/1.1	28	0.9946	[0.9933, 0.9960]	[0.9924, 0.9961]	equivalent within [0.98, 1.02]
C1	L, io_uring	h2c	28	0.9854	[0.9832, 0.9886]	[0.9822, 0.9893]	equivalent within [0.98, 1.02]
C1	L, io_uring	TLS	28	0.9982	[0.9970, 1.0002]	[0.9961, 1.0016]	equivalent within [0.98, 1.02]
C1	L, io_uring	MQTT	28	0.9951	[0.9938, 0.9964]	[0.9927, 0.9973]	equivalent within [0.98, 1.02]
C1	W, IOCP	HTTP/1.1	28	0.9978	[0.9951, 0.9995]	[0.9884, 1.0006]	equivalent within [0.98, 1.02]
C1	W, IOCP	h2c	0	none	none	none	outside the confirmatory cost family: generator-bound on W
C1	W, IOCP	TLS	28	0.9969	[0.9928, 1.0039]	[0.9911, 1.0067]	equivalent within [0.98, 1.02]
C1	W, IOCP	MQTT	0	none	none	none	outside the confirmatory cost family: generator-bound on W
C2	L, epoll	HTTP/1.1	28	0.9985	[0.9977, 1.0007]	[0.9973, 1.0010]	equivalent within [0.98, 1.02]
C2	L, epoll	h2c	28	0.9997	[0.9973, 1.0043]	[0.9969, 1.0048]	equivalent within [0.98, 1.02]
C2	L, epoll	TLS	28	0.9979	[0.9958, 1.0039]	[0.9946, 1.0049]	equivalent

The two cells in Holm that fail are C3's h2c cells on L. On epoll the median ratio of the time to first byte is 1.0263, and its 95% interval, [1.0244, 1.0280], lies wholly above 1.02. So the analysis reports a measured cost, against one-port mode. Only 2 of its 28 sessions lie below the upper margin. On io_uring the median is 1.0212, with the interval [1.0197, 1.0232]. Its lower end lies inside the margin, so the verdict is that equivalence is not shown, and no measured cost is claimed there. 9 of its 28 sessions lie below the upper margin. The C1 and C2 cells of h2c pass on both Linux backends. The joint power the pilot gave h2c's three cells was 1.000 on epoll and 1.000 on io_uring, so these failures are not a matter of low power.

The equivalence claim needs C1, C2 and C3 to pass for one protocol, backend and host. It holds for 8 configurations. They are HTTP/1.1 on epoll (L), TLS on epoll (L), MQTT on epoll (L), HTTP/1.1 on io_uring (L), TLS on io_uring (L), MQTT on io_uring (L), HTTP/1.1 on IOCP (W) and TLS on IOCP (W). The joint power that the pilot gave each of these claims is 1.000 on L, except for MQTT on io_uring with 0.993. On W it is 0.790 for HTTP/1.1 and 0.848 for TLS. As stated before the run, the TLS cells have little power to show the cost of a check of 6 bytes. So their passes are not read as evidence about that check.

On W, h2c and MQTT have no claim. Their C1 cells are outside the confirmatory family, since W's churn of these two protocols measures the generator (Section 6.1). The pilot's rows are consistent with that. Over the 64 pilot windows of each cell, the server's CPU was busy for a median of 39.8% in h2c and 34.7% in MQTT. It was busy for 96.2% in HTTP/1.1, whose server closes first. The generator's CPUs were 44.3% and 44.6% busy, so neither side was saturated. Development diagnostics traced the limit to the generator's bind and close, which Windows serializes across its threads. Their C3 cells were not resolved by the pilot. They ran at R_C sessions and are reported as not resolved, with no equivalence claimed. h2c has the median 0.9906 with the interval [0.9582, 1.0131], and MQTT 1.0183 with [0.9889, 1.0360]. Their open-loop rates, 0.5 times W's generator-bound churn medians, were 2,496.680 exchanges per second for h2c and 2,609.799 for MQTT, against 8,544.537 for HTTP/1.1. So on W they ran at a smaller share of the server's capacity than open HTTP/1.1. Their C2 cells pass.

No cost window on L was invalid. On W the C3 cells of HTTP/1.1, h2c and MQTT have 1, 1 and 1 invalid windows; Table A.12 gives their arms and reasons. The analysis clustered by lab job decides nothing. Only W's cost cells ran in more than one job, and their clustered intervals are in Table A.14. With two jobs such an interval spans little more than the two jobs' medians, so it carries little information.

The secondary metrics of WL4 are in Table A.13. In every C1 and C3 cell on L, one-port mode's median server CPU per exchange was above dedicated mode's. In C1's HTTP/1.1 cell on epoll, for example, it was 1.0137 times, with the interval [1.0130, 1.0147]. In C2 the medians lie on both sides of one. The tail of the time to first byte follows the median in C3's h2c cells on L: the ratio of the 99th percentile is 1.0255 on epoll and 1.0219 on io_uring.

5.2. Robustness

B1 and B2. The hard-case runners ran every case of the frozen table, 16 times, on each entry of backend, detection mode and dispatch: 8 entries on L and 2 on W. That is 20,992 runs on L and 5,248 on W, and all 26,240 of the 26,240 runs passed. They hold 0 deviations from the frozen table and 0 failed checks of B2. Every server check at the end of an entry was in order: 52 of 52 on L and 12 of 12 on W. The analysis lists 0 measured windows that counted a connection classified other than as its script's protocol. Its count covers the windows that record one, the one-port windows with in-process dispatch. In the other measured windows, a check of the server's counters per class after the analysis found only each script's protocol and the probe. So B1 holds and B2 holds. In 11,456 runs on L and 2,939 on W the detection woke more than once before it decided.

B2's distributions. Table 7 gives the lateness of timed events and the decision latency per backend, which the frozen text reports and does not test. They come from the same runs, and every run of a backend that holds a value counts, whatever its case, detection mode or dispatch. The lateness is the time from a timed event's deadline to the return of the wait that the loop handled it after. Its median was 3.249 ms on epoll, 0.289 ms on io_uring and 5.476 ms on IOCP. The largest lateness, 3.548, 0.732 and 16.109 ms, lies below the guard band G of each host, 4 ms on L and 17 ms on W. The decision latency runs from the client's last write before the end of detection to that end, on one clock. In the runs no timer ended, its median was 16.3 μs on epoll, 16.2 μs on io_uring and 28.5 μs on IOCP. These runs include HC21 and HC22 with a partial signature, whose detection ends on the client's half-close or reset, a pause after its last write. There are 192 such runs on each Linux backend and 96 on IOCP. Their latency holds that pause: at least 5,057.8 μs on epoll, 5,050.9 μs on io_uring and 5,081.8 μs on IOCP. On the Linux backends they form the upper tail: every other run took at most 902.6 μs on epoll and 943.4 μs on io_uring. On IOCP 5 other runs took longer than any of them, in HC2, HC9, HC10 and HC23, the longest 151,299.7 μs. In the runs a timer ended, detection ends when the loop handles the timer, so the latency holds the time from the client's last write to that handling.

Table 7: B2's distributions per backend, over the hard-case runs: L's 10,496 runs on each Linux backend and W's 5,248 on IOCP. The lateness is that of each timed event past its deadline, in the runs a timer ended. The decision latency runs from the client's last write before the end of detection to that end. A run with no such write has none: 384 per backend on L and 192 on W. Its column gives the 99th percentile, the value at rank $\lceil 0.99\,n \rceil$ of the sorted values.

Backend	Distribution (unit)	Runs	Min	Median	99th pct.	Max
epoll	lateness of a timed event (ms)	704	0.073	3.249	3.394	3.548
epoll	decision latency, runs no timer ended (μs)	9,600	11.0	16.3	5,103.6	5,122.7
epoll	decision latency, runs a timer ended (ms)	512	121.996	875.100	3,003.427	3,003.470
io_uring	lateness of a timed event (ms)	704	0.028	0.289	0.422	0.732
io_uring	decision latency, runs no timer ended (μs)	9,600	10.4	16.2	5,104.0	5,131.2
io_uring	decision latency, runs a timer ended (ms)	512	121.909	873.028	3,000.412	3,000.953
IOCP	lateness of a timed event (ms)	352	0.001	5.476	15.649	16.109
IOCP	decision latency, runs no timer ended (μs)	4,800	11.9	28.5	5,126.2	151,299.7
IOCP	decision latency, runs a timer ended (ms)	256	0.049	875.482	3,014.620	3,016.206

B3. Table 8 gives every cell. Of the 18 cells in Holm, 14 pass under both computations, which disagree on 0 cells. In each pass the competitor held at least 1.10 times the server's counted footprint per pending connection, by B3's test. The passes are nginx, HAProxy and Envoy in both cases, and hyper-util in the silent case, each on epoll and on io_uring. Each comes with its difference D, the competitor's W less the server's, in bytes per pending connection. The passes' medians of D range from 1,837.8 bytes against hyper-util on io_uring to 10,726.3 against Envoy in the partial-ClientHello case on epoll. The server's own bounds for a pending connection are those of Section 3.5. It holds no data buffer before the first byte, the handler's buffer in replay, and at most B_{CH} of a ClientHello in pass-through.

Table 8: B3, competitor / server, on L. Q is the ratio of counted footprints, with its median, 95% interval and interval at Holm's level; X counts the sessions with Q above 1.10. D is the difference in bytes per pending connection, with its 95% interval. "Shown" stands for the analysis's verdict that the competitor held at least 1.10 times the server's counted footprint per pending connection, by B3's test.

Case	System	Backend	Valid	Q	95% interval	Holm interval	X	D	95% interv
silent	nginx	epoll	16	1.6728	[1.6665, 1.6820]	[1.6580, 1.6883]	16	4,491.1	[4,473.4, 4,5
silent	nginx	io_uring	16	1.6275	[1.6201, 1.6307]	[1.6163, 1.6335]	16	4,288.6	[4,273.7, 4,3
silent	HAProxy	epoll	16	1.3960	[1.3899, 1.3986]	[1.3889, 1.4019]	16	2,669.5	[2,651.3, 2,6
silent	HAProxy	io_uring	16	1.3641	[1.3607, 1.3672]	[1.3577, 1.3714]	16	2,502.7	[2,481.3, 2,5
silent	Envoy	epoll	16	2.4496	[2.4399, 2.4626]	[2.4357, 2.4659]	16	9,679.5	[9,671.7, 9,6
silent	Envoy	io_uring	16	2.3733	[2.3647, 2.3852]	[2.3580, 2.3934]	16	9,504.3	[9,488.6, 9,5
silent	sslh-ev	epoll	16	0.9985	[0.9958, 1.0000]	[0.9948, 1.0002]	0	−9.8	[−27.9, −0.
silent	sslh-ev	io_uring	16	0.9722	[0.9702, 0.9746]	[0.9699, 0.9762]	0	−190.6	[−209.1, −1
silent	hyper-util	epoll	16	1.3008	[1.2991, 1.3051]	[1.2977, 1.3063]	16	2,017.6	[2,003.4, 2,0
silent	hyper-util	io_uring	16	1.2675	[1.2632, 1.2732]	[1.2624, 1.2738]	16	1,837.8	[1,819.0, 1,8
partial ClientHello	nginx	epoll	16	1.7906	[1.7817, 1.7934]	[1.7771, 1.7954]	16	5,564.0	[5,551.7, 5,5
partial ClientHello	nginx	io_uring	16	1.7447	[1.7389, 1.7495]	[1.7363, 1.7499]	16	5,414.8	[5,389.7, 5,4
partial ClientHello	HAProxy	epoll	16	1.9177	[1.9119, 1.9220]	[1.9109, 1.9229]	16	6,509.8	[6,490.5, 6,5
partial ClientHello	HAProxy	io_uring	16	1.8707	[1.8679, 1.8749]	[1.8546, 1.8770]	16	6,345.7	[6,327.7, 6,3
partial ClientHello	Envoy	epoll	16	2.5069	[2.4950, 2.5195]	[2.4888, 2.5219]	16	10,726.3	[10,709.2, 1
partial ClientHello	Envoy	io_uring	16	2.4276	[2.4241, 2.4454]	[2.4214, 2.4475]	16	10,544.0	[10,522.4, 1
partial ClientHello	sslh-ev	epoll	16	0.9596	[0.9572, 0.9629]	[0.9572, 0.9629]	0	−284.0	[−304.3, −2
partial ClientHello	sslh-ev	io_uring	16	0.9398	[0.9364, 0.9432]	[0.9359, 0.9441]	0	−437.4	[−464.5, −4
silent	Netty	epoll	16	1.1887	[1.1852, 1.1933]	none	16	1,283.0	[1,262.0, 1,3
silent	Netty	io_uring	16	1.1617	[1.1578, 1.1635]	none	16	1,124.6	[1,104.7, 1,1
silent	Jetty	epoll	16	1.7930	[1.7846, 1.8176]	none	16	5,307.4	[5,251.9, 5,3
silent	Jetty	io_uring	16	1.7777	[1.7688, 1.8017]	none	16	5,285.1	[5,189.6, 5,3
silent	caddy-l4	epoll	16	2.5698	[2.5645, 2.5894]	none	16	10,570.3	[10,553.5, 1
silent	caddy-l4	io_uring	16	2.5227	[2.5182, 2.5371]	none	16	10,415.8	[10,389.3, 1
silent	cmux	epoll	16	1.4936	[1.4907, 1.5032]	none	16	3,328.9	[3,306.7, 3,3
silent	cmux	io_uring	16	1.4626	[1.4567, 1.4683]	none	16	3,174.2	[3,125.9, 3,1
partial ClientHello	Netty	epoll	16	0.7979	[0.7957, 0.7998]	none	0	−2,211.8	[−2,235.8, -
partial ClientHello	Netty	io_uring	16	0.7852	[0.7833, 0.7876]	none	0	−2,384.7	[−2,396.9, -
partial ClientHello	Jetty	epoll	16	3.5742	[3.5644, 3.5918]	none	16	27,601.7	[27,524.5, 2
partial ClientHello	Jetty	io_uring	16	3.5225	[3.5123, 3.5492]	none	16	27,767.0	[27,555.4, 2
partial ClientHello	caddy-l4	epoll	16	2.8579	[2.8400, 2.8668]	none	16	13,121.6	[13,103.7, 1
partial ClientHello	caddy-l4	io_uring	16	2.7724	[2.7625, 2.7957]	none	16	12,912.2	[12,881.1, 1
partial ClientHello	cmux	epoll	15	none	none	none	none	none	none
partial ClientHello	cmux	io_uring	16	1.7127	[1.7061, 1.7150]	none	16	7,833.4	[7,812.4, 7,8

Every pass holds for W as configured and with WL7's exclusions. Outside W are the reserved but unused part of each socket's receive memory and kernel memory outside the slab that no receive queue is charged for. The co-tenants' growth of the merged cache is outside it too. In the partial-ClientHello case nginx and Envoy left bytes queued, with K_q of 1,068.0 and 1,068.0 bytes per pending connection on epoll and the same on io_uring. So the two residuals of Section 4.2.1 apply to their passes. The server, in replay, left none queued. Tables A.17 and A.18 give the parts of every W, the caches' growth and the field f.

The 4 other cells in Holm are sslh-ev's, and each is a loss: the 95% interval of

Q lies wholly below one. sslh-ev's counted footprint per pending connection was smaller than the server's. In the silent case D is -9.8 bytes on epoll, with the interval $[-27.9, -0.2]$, and -190.6 on io_uring, with $[-209.1, -173.0]$. On epoll the upper end of Q's interval prints as 1.0000 at four places, below one before rounding. At Holm's level that cell's interval, [0.9948, 1.0002], contains one; the frozen definition of a loss reads the 95% interval. In the partial-ClientHello case D is -284.0 on epoll, with $[-304.3, -263.0]$, and -437.4 on io_uring, with $[-464.5, -409.2]$.

K_{BASE} is 6,463.9 bytes per pending connection: the median of `ophold`'s K_s over its valid windows. Every system pays it alike, and it is most of the server's W, which is 6,677.2 bytes in the silent case against nginx on epoll. There the server's U is 407.1 bytes, and its $K_s - K_{\mathrm{BASE}}$ is -193.8 bytes. K_s falls below K_{BASE} in many arms, every arm on epoll among them, so K_{BASE} is a shared reference, not a floor.

B3's 16 descriptive cells are reported against the same bound and decide nothing. Netty held less than the server in the partial-ClientHello case, with Q of 0.7979 on epoll and 0.7852 on io_uring and D of $-2{,}211.8$ and $-2{,}384.7$ bytes. Every other descriptive cell with an interval lies above the bound. One cell, cmux's partial-ClientHello cell on epoll, ended with 15 valid sessions after its reruns, short of R, and so has no interval. Its 5 invalid windows each found the host's count of TIME-WAIT sockets changed at a sample.

By the narrowing order, the B3 claim is narrowed to the competitors and cases it beats. These are nginx, HAProxy and Envoy in both cases and hyper-util in the silent case, on both Linux backends.

5.3. Mechanisms

Table 9 gives every cell. Of the 20 cells, 15 pass under both computations, which disagree on 0 cells.

Table 9: The mechanism family. M1 is default / other detection mode in connections per second. M2 is in-process / relay in CPU per connection (WL6). M3 is the server's relay / the proxy in connections per second.

Hyp.	Host	Backend, protocol, system	Valid	Median	95% interval	Verdict
M1	L	epoll, HTTP/1.1	16	1.0320	[1.0301, 1.0328]	shown
M1	L	io_uring, HTTP/1.1	16	1.0557	[1.0540, 1.0588]	shown
M1	W	IOCP, HTTP/1.1	16	1.0371	[1.0331, 1.0433]	shown
M1	L	epoll, h2c	16	1.0273	[1.0237, 1.0311]	shown
M1	L	io_uring, h2c	16	1.0385	[1.0359, 1.0408]	shown
M1	W	IOCP, h2c	16	0.9524	[0.8837, 1.0773]	a difference was not shown
M2	L	epoll, HTTP/1.1	16	0.2726	[0.2714, 0.2731]	shown
M2	L	io_uring, HTTP/1.1	16	0.2475	[0.2458, 0.2485]	shown
M2	L	epoll, TLS	0	none	none	a difference was not shown; untested: 0 valid sessions, fewer than R = 16
M2	L	io_uring, TLS	0	none	none	a difference was not shown; untested: 0 valid sessions, fewer than R = 16
M3	L	epoll, TLS (SNI), nginx	16	1.0865	[1.0854, 1.0890]	shown
M3	L	epoll, TLS (SNI), HAProxy	16	1.0988	[1.0974, 1.1005]	shown
M3	L	epoll, TLS (SNI), Envoy	16	2.1993	[2.1891, 2.2032]	shown
M3	L	epoll, TLS (SNI), caddy-l4	16	2.1118	[2.1089, 2.1165]	shown
M3	L	epoll, TLS (SNI), sslh-ev	0	none	none	not shown; untested: 0 valid sessions, fewer than R = 16
M3	L	epoll, HTTP/1.1, nginx	16	1.0804	[1.0778, 1.0838]	shown
M3	L	epoll, HTTP/1.1, HAProxy	16	1.1157	[1.1149, 1.1171]	shown
M3	L	epoll, HTTP/1.1, Envoy	16	2.1723	[2.1673, 2.1939]	shown
M3	L	epoll, HTTP/1.1, caddy-l4	16	2.0485	[2.0445, 2.0500]	shown
M3	L	epoll, HTTP/1.1, sslh-ev	0	none	none	not shown; untested: 0 valid sessions, fewer than R = 16

M1. M1 passes in 5 of its 6 cells. Replay, the default on every backend, served more connections per second than peek for HTTP/1.1 on all three backends and for h2c on epoll and io_uring. For h2c on IOCP a difference was not shown: the median is 0.9524, with the interval [0.8837, 1.0773]. On IOCP, peek is a zero-byte receive followed by a synchronous peek. An undecided peek would switch to replay, since Windows has no low-water mark. None did: the counters show 0 switches in 3,159,269 connections of the valid peek windows for HTTP/1.1, and 0 in 1,121,373 for h2c. So the IOCP cells compare replay with that two-step peek, and measure the interface as much as the idea.

M2. M2 passes in 2 of its 4 cells, the two cells of HTTP/1.1. In-process dispatch used 0.2726 times the CPU per connection of relay on epoll and 0.2475 times on io_uring, at the rates 6,887.012 and 7,408.660 exchanges per second. The two TLS cells were not run. In closed loop the back end that terminates TLS limits the relay arm, so its relay windows broke the rule for back-end cores. Development runs showed this before the freeze, and the author decided then not to run the two cells. The rate sessions after the pilot entry ended the same way, so the frozen rule computes no rate for them. Each cell enters Holm with $p = 1$, and a difference was not shown. M2's claim narrows to HTTP/1.1.

M3. M3 passes in 8 of its 10 cells. The server's relay served more connections per second than nginx, HAProxy, Envoy and caddy-l4, for TLS routed by its server name and for HTTP/1.1. The medians range from 1.0804 against nginx for HTTP/1.1 to 2.1993 against Envoy for TLS. The two cells of sslh-ev are untested. Every one of their sslh-ev windows broke the error rule, as the defect that the revision log records before the freeze predicts, and some also had failed connects. In development the defect showed no failed connect and no listen overflow, and the cause of these is not established. sslh-ev's listen overflow counter rose by a total of 46 and 51 in the two cells; no other proxy's rose. By the narrowing order, M3 is narrowed to the four proxies it beats.

Secondary mechanism cells. At a fixed load, replay's median time to first byte was below peek's in every M1 cell (Table A.16). The interval lies wholly below one in all but the h2c cell on IOCP. Against the proxies the server's relay used less CPU per connection in every cell, with medians from 0.8990 against nginx to 0.3515 against Envoy. Its median time to first byte at a fixed load was higher than nginx's, 1.0182 times for HTTP/1.1 and 1.0142 for TLS, and lower than the other three proxies'.

5.4. The Secondary Cells

Section 4.9 lists the secondary cells; Tables A.15 and A.16 give each, and none decides anything. The results files list 19 cells of any kind short of their R, each with its logged reason: 8 were not run, and 11 ran without reaching R. Section 6 states why the secondary cells ran again on L.

SSH. Under churn the one-port arm served 0.9863 and 0.9849 times the dedicated arm's connections per second on L, and 1.0077 on W. At a fixed load its median time to first byte was 1.1105, 1.0642 and 1.1036 times dedicated mode's. The message order differs by mode, as stated before the run: in one-port mode the server's identification line waits for the client's. The 3 SSH cells

of C2 could not run. The generator has no keep-alive form of SSH, and the server's SSH handler closes after one line. Each cell ran its sessions and its reruns, and all 80 of its windows were invalid. On L the archived job sl1 had shown the same (Section 6.2).

The mixed-protocol cell. On epoll the one-port arm served 0.9958 times the dedicated arm's connections per second, with the interval [0.9938, 0.9971]. On IOCP it served 0.9939 times, with [0.9871, 0.9995]. By design the background's silent connections live differently in the two modes. In one-port mode the listener has no fallback, so the server closes each at T_{dec} and the holder opens it again at once. In dedicated mode they are held for the whole window. The archived job sl1 showed this in its counters. So the background's make-up is the same in both modes, and its connections' lives are not.

On io_uring the cell has no valid session. It ran 20 sessions and 80 windows, of which 0 were valid, in both arms alike. The probe of HTTP/1.1 timed out in 76 windows, and the share of failed exchanges exceeded its bound in 80. The churn starved beside its keep-alive background in both modes, as sl1 had shown, so this is not a cost of detection. We report it as a measured limitation of the server's io_uring backend. That backend's receives take buffers that the kernel selects from a ring, which holds 128 free buffers per worker and gets a new one each time the worker takes one. A receive that finds the ring empty fails and is posted again. The cell's background keeps 64 TLS and 64 MQTT connections busy. How these might starve the churn is a hypothesis, stated in Section 7.2.

TLS variants. With ALPN `h2`, the one-port arm served 0.9957, 0.9970 and 0.9942 times the dedicated arm's connections per second on epoll, io_uring and IOCP. TLS with session resumption was not run, for the reason given in Section 4.9.

Two server cores. With two cores the one-port arm served 0.9895 and 0.9942 times the dedicated arm's connections per second on epoll and io_uring. The cell on IOCP was not run: the server serves one worker on IOCP, and more would be a different threading model, outside this paper. A `SO_REUSEPORT` group of two listeners served 1.0024 times the shared listener's connections per second on epoll, with the interval [0.9919, 1.0064]. On io_uring it served 1.9727 times, with [1.9350, 1.9938].

The relay on io_uring. Against the four proxies with valid sessions, the server's relay on io_uring served from 1.1676 times nginx's connections per second, for HTTP/1.1, to 2.4016 times Envoy's, for TLS. sslh-ev's 4 secondary cells ended with no valid session, as its M3 cells did and as the follow-up of sl1 expected. Its two cells of the time to first byte at a fixed load ran closed-loop windows instead, since M3 gave them no rate.

IOCP's forms. AcceptEx with a receive buffer served 1.0756 times the default form's connections per second, with [1.0725, 1.0812]. The posted receive buffer served 1.0058 times, with [0.9982, 1.0131]. The posted form holds a buffer for a connection that has received no byte, so it breaks B2's memory bound.

B3 in the other detection mode. With the server in peek, its W was 0.9994 and 0.9871 times its W in replay in the silent case, on epoll and io_uring. In the partial-ClientHello case it was 1.1223 and 1.1013 times. Peek leaves the

received bytes queued in the kernel, where K_q counts them.

Counts. The operations and copies per connection, the system calls of the proxies and the check of the server's system-call counters are in Section 5.5.

5.5. Operations, Copies and System Calls

These outputs are counts. None is timed and none decides anything. Two sources hold them. The measured windows hold the server's counters over each window's process, and Section 3.9 defines them. An untimed lab job, st1, ran after the analysis on L's measured build. It ran the frozen tool for these windows, after the freeze guard, and holds counts only (Section 6.7). Its churn is open loop at 2,000 exchanges per second for 2.0 s after 0.5 s, and its keep-alive uses 2 connections for 0.5 s. C1 and C3 share that churn, so their untimed windows differ only in name.

The copy counters count bytes, not buffers. "Copied in user space" counts each move of a payload byte by the server's own code from one user-space place to another. The bytes that a system call moves count as received, sent or peeked. Producing a byte, as a handler, OpenSSL or nghttp2 does, is not a copy.

Operations and copies, in-process. Appendix Tables A.19 and A.20 give, per arm, the means over the valid windows of each window's counts, per connection, and per request in C2. Table A.19 gives operations, bytes copied in user space and bytes received and sent; Table A.20 adds bytes peeked and wakeups during detection. Operations are the system calls and ring submissions the counters name. In replay the counter of bytes copied in user space recorded none in any arm of the 34 cost cells, and none was peeked. The largest means are 0.0 and 0.0 bytes. One-port mode's operations differed from dedicated mode's by at most 0.077 per connection in C1 and C3 and 0.001 per request in C2. In M1, peek copied 24.0 bytes per connection with `MSG_PEEK`, which stay queued in the kernel, on every backend.

Operations and copies, relay. No measured window runs the relay in peek. So both detection modes of the relay come from st1, under one load, per relayed connection (Appendix Table A.21). Relay dispatch has no counterpart in dedicated mode, so peek is read against replay. These counts depend on the load. At a low rate each step waits on its own, so the loop's waits and the accepts that find no connection are more per connection. They are therefore not those of M2's and M3's relay windows, which ran at other loads. The relay reads into its buffers and sends from them, so each relayed byte counts once as received and once as sent. Its counter of bytes copied in user space recorded none in either mode on either backend: the largest mean is 0.0. In peek it peeked 24.0 bytes per relayed connection for HTTP/1.1 and 235.0 for TLS on epoll. Its operations per relayed connection for HTTP/1.1 were 17.712 in replay and 19.301 in peek on epoll, and 21.000 and 24.001 on io_uring.

System calls of the proxies. Table 10 gives each proxy's system calls per connection, from `perf trace -s`, in front of the stub as in M3, with the server's relay beside them. For HTTP/1.1 nginx made 22.022 calls per connection, HAProxy 44.826, Envoy 43.945, caddy-l4 38.446 and sslh-ev 21.979.

The server's relay made 20.707. For the calls that the server's checks name, `perf stat`'s count stands beside that of `perf trace -s`. In every row `perf trace -s` counted fewer of them, by 0.02% to 0.12%, and reported 0 events lost.

The counter check. The untimed windows cover the 24 cost cells of L in both modes. In them, 360 of the 360 checks of the server's counters equal `perf stat`'s count of the same calls. Appendix Table A.22 gives them per cell. That is the check the frozen text asks for, read before the freeze as a comparison with `perf stat`. `perf trace -s` agreed in 297, short of `perf stat` by at most 0.42% of a call's count, with 0 events reported lost. In the relay's 8 rows, 60 of 60 checks equal `perf stat`'s count.

Table 10: The proxies' system calls per connection on L, from st1's untimed windows: open-loop churn at 2,000 exchanges per second, in M3's arrangement with the stub. "All" counts every call that `perf trace -s` saw, over the connections the stub accepted. "Checked" counts only the calls the server's checks name, by `perf trace -s` and by `perf stat`; the shortfall is their difference over `perf stat`'s count.

System	Protocol	Conns.	All	Checked, trace	Checked, stat	Shortfall %	Load ended ok
nginx	TLS (SNI)	5,002	22.542	15.543	15.560	0.11	yes
nginx	HTTP/1.1	5,002	22.022	15.024	15.032	0.05	yes
HAProxy	TLS (SNI)	5,002	44.887	18.942	18.957	0.07	yes
HAProxy	HTTP/1.1	5,002	44.826	18.909	18.925	0.09	yes
Envoy	TLS (SNI)	5,002	43.991	31.992	32.011	0.06	yes
Envoy	HTTP/1.1	5,002	43.945	31.950	31.965	0.05	yes
caddy-l4	TLS (SNI)	5,002	38.209	23.027	23.035	0.04	yes
caddy-l4	HTTP/1.1	5,002	38.446	23.115	23.119	0.02	yes
sslh-ev	TLS (SNI)	5,002	21.979	8.989	9.000	0.12	yes
sslh-ev	HTTP/1.1	5,002	21.979	8.991	9.000	0.10	yes
the server's relay	TLS (SNI)	5,002	20.855	17.854	17.860	0.03	yes
the server's relay	HTTP/1.1	5,002	20.707	17.706	17.712	0.03	yes

5.6. The Competitors' Hard Cases

The competitors' table runs every hard case that a system's listener setup covers, at matched timers and at the system's defaults, 3 replicates each. It is descriptive and decides nothing, and no time from it is reported. Each row records whether the reply equals what the server's frozen table expects. That field is neither a pass nor a failure of the system, whose own specification may differ from the server's table.

The table holds 8,052 rows from 9 systems, over 164 variants of 25 cases. Of the 7,977 rows observed, the reply of 3,327 equals what the server's frozen table expects, and the reply of 4,650 does not. 75 rows were not observed, since the case generator did not end. 243 observed rows reached no decision within 60 s and are counted by their reply. The replicates agreed within every variant: 0 variants are mixed. Table 11 gives the counts per system.

Table 11: The competitors' hard cases, per system, on L. A variant is one case's form at one timer setting with its replicates; it counts as equal or differing only when every replicate does. "Equal" means the reply equals what the server's frozen table expects.

System	All	Rows Equal	Rows Differ	Not obs.	Variants	Matched timers Equal	Matched timers Differ	Default timers Equal	Default timers Differ	Cases not covered
nginx	948	390	558	0	158	65	93	65	93	HC5, HC7, HC15
HAProxy	984	549	435	0	164	123	41	60	104	none
Envoy	984	693	291	0	164	116	48	115	49	none
caddy-l4	948	714	234	0	158	119	39	119	39	HC5, HC7, HC15
sslh-ev	774	492	264	18	129	81	46	83	42	HC9, HC10, HC11, HC12, HC13, HC14, HC15
Netty	948	126	807	15	158	24	134	18	135	HC5, HC7, HC15
Jetty	948	66	882	0	158	11	147	11	147	HC5, HC7, HC15
cmux	774	273	483	18	129	48	80	43	81	HC9, HC10, HC11, HC12, HC13, HC14, HC15
hyper-util	744	24	696	24	124	4	116	4	116	HC5, HC7, HC9, HC10, HC11, HC12, HC13, HC14, HC15

6. Deviations and Reproducibility

Every change after the hypotheses freeze is in the revision log, with its reason. Each change to the frozen code or rules was logged before any data it governs. Three entries came after the data they concern: the escaping of Section 6.4, which changes no decision, and the readings of Sections 6.3 and 6.7. This section lists the changes and records that bear on the results, in the order they were made.

6.1. Decisions Made before the Data

L's kernel changed before the code freeze. The frozen text fixes L at kernel 7.2.3. By the author's decision L moved to 7.2.6-arch2-1 by kexec, so that

the module that exempts loopback traffic from connection tracking could load. Every cited source line was checked unchanged at the new kernel's tag, and the records, the pilot and every confirmatory run used 7.2.6-arch2-1.

Four more decisions came before the code freeze, each before any confirmatory data. First, on W the churn of h2c and MQTT measures the generator, not the server, so the author placed their C1 cells outside the cost family. In both protocols the client closes first, so each connection leaves a TIME-WAIT socket that holds a client port. Development runs showed Windows' bind and close in the generator growing slower as those sockets grew. The two cells ran in the pilot only, which gives the open-loop rates of the C3 cells of h2c and MQTT on W. The pilot's rows show neither the server nor the generator's CPUs saturated in them (Section 5.1). Second, the author decided not to run M2's TLS cells (Section 5.3). Third, the author decided not to run the secondary cell with two server cores on IOCP, since the server serves one IOCP worker. Fourth, every job of the session engine starts with discarded windows of at least 80 s. Two development jobs on W showed a slow start of the job, of unknown cause, that lasted several windows. The hard cases, the windows of K_{BASE} and the pilot's parts run outside that engine and have no such warm-up.

After the pilot entry, rule E kept every proposed default, and M2's rates were set from development sessions. Section 4.7 gives both.

6.2. The Count of Misclassified Connections, a Later Change

The defect. In the one-port arm of the mixed-protocol cell, the runners bounded each script's class in the count of misclassified connections. They read each bound from a field that the generator's report does not hold. So the bounds were the probe's one connection for HTTP/1.1 and none for TLS and MQTT. Every connection of the window but one then counted as classified other than as its script's protocol. By the rule of validity, every one-port window of the mixed cell would be invalid. And the analysis's list of B1 failures would list each such window, after which the robustness claim is withheld.

The evidence. The first job of the secondary cells on L, sl1, stopped after 384 windows, 331 of them valid. It ran 14 one-port windows of the mixed cell, 8 on epoll and 6 on io_uring. In each, the server's counters classified as HTTP/1.1 exactly the generator's connections and the probe, and as TLS and MQTT exactly the background's connections. The row's count was the generator's connections plus 128. In one row on epoll the generator made 105,736 connections. The server classified 105,737 as HTTP/1.1, 64 as TLS and 64 as MQTT, and the row counted 105,864 misclassified. With the bounds that the generator does report, the count in each of these windows is zero. The counters are totals per class, so neither count can see two connections whose classes are swapped.

The change. The author followed the frozen rule for a later change. One commit, `9cae2deff2b01896bfab25c5af4c610381dc3c1d`, changed five Python files of the runners, their tests and the freeze guard, and nothing that is compiled. On W every target's hash of inputs equals the measured build's at the code freeze. On L the change touches no file that a target compiles. So the green

sanitizer records and the gates of the code freeze still cover every measured build, and no record was made again. The rule archives every confirmatory window of an old build. Read with the build as the compiled build, which the records cover, it found no such window, since no compiled input changed. The pilot and its entry stand, as do rule E's choices and M2's rates.

What was run again. The scope of what ran again was the author's decision, not the rule's. All of sl1's windows are archived and kept only as the defect's evidence, and no analysis reads them. The secondary cells ran again from their start on L and for the first time on W, and the competitors' table ran, all with the changed runners. The four secondary cells of B3 in the other detection mode had run in B3's own job, whose rows hold no such count, and were not run again. Every row of the new jobs names the change's commit. Beside the defect, the follow-up of sl1 recorded three readings and one expectation, each named beside the new rows in Section 5.4. The mixed cell on io_uring starved in both modes, the SSH cells of C2 could not run, and the silent connections' lives differed by mode. sslh-ev's secondary cells were expected to end without a valid session. No measured window of the new jobs counted a misclassified connection.

6.3. B1 and B2 from the Runners' Rows

The frozen text has the analysis code compute every decision. That code reads no row of the hard cases, so for B1's hard cases and for B2 it computes nothing. A reading in the revision log, made after those runs and before any count reached the paper, takes both decisions from the runners' rows. The runners judge each run with the frozen test suite's checks, ported line for line. A separate generator counts the rows, names each input by its SHA-256, and stops on any row that disagrees with itself. So for these two decisions the frozen sentence does not hold in its letter.

6.4. The Macros' Escaping, a Later Change

The analysis's macro writer wrote text taken from the summary as it was. Of its 2,168 macros, 6 held a percent sign or an underscore, which stopped every build of the paper, so the file was held back. One commit made the writer escape TeX's special characters. The code that computes the decisions did not change. The checks were defined in the revision log before they ran. On W, the analysis run again from a fresh clone wrote the same decisions byte for byte, as required. Its summary differed only in the commit it records, and its macros only in those lines.

On L, with Python 3.14.7 against W's 3.14.5, the decisions file was not byte-identical to W's. The provenance of the macros, among the results files, records the counts and largest values of the differences and names the full comparison by its SHA-256. Every verdict, pass, count, rank, median and interval bound is equal. The floats that differ are BCa p-values, their Holm adjustments and one jackknife acceleration. The largest relative difference in the decisions file is 4.1×10^{-16}, and the largest absolute difference in the summary is 1.1×10^{-16}. The two hosts wrote the same macros byte for byte, so no printed value differs.

6.5. W's Procedure

The frozen procedure on W asks for a CPU frequency counter tested against a known load. No counter passed: under three power plans neither the counter nor a fixed loop's rate of work moved. So W's windows were validated without the frequency rule, and each row records the counter. W's CPU measure of WL4 is the server's processor cycles, read with `QueryProcessCycleTime`, a design choice made before any confirmatory data. Windows' process times move in whole clock ticks of 15.625 ms, so at a low load they sample the CPU time rather than sum it. A ratio of two arms' cycles needs no conversion. W has no counter of listen overflows. A connection it refuses shows as a failed connect, which makes the window invalid. On W the generator binds each source address with `SO_REUSE_UNICASTPORT`, the reading of Section 3.10.

The check that a W job must pass before it starts was lowered by the author's decision, before any pilot data. It now asks for a mean idle of at least 90%. Each CPU a window uses must be at least 90% idle, and no process may use more than 10% of a CPU. It had asked for 95%, 95% and 5%. With programs the author keeps running on W, it had refused job starts through an evening. The check gates a job's start only, and every rule of validity of a window is unchanged.

The freeze guard reads the frozen text through git in the locale's encoding, a Windows code page on W, while the text is UTF-8. So the first start of W's pilot stopped in the guard, before any window. Every frozen run on W then ran in Python's UTF-8 mode, set for its own process only. Every text that mode changes is ASCII, except git's, so no recorded value depends on it. The guard itself is not fixed; that would be a later change.

The cost family on W ran as two jobs. The first was stopped at the author's request, inside the first window of a session; that window is invalid with the stop as its reason. The second job resumed from the rows already written. At the end of the family's order it ran a new session in place of the cut one, as the rule for invalid windows prescribes.

6.6. The Analysis Code

Before the code freeze the analysis code moved once. Its first named commit, `7364fbbbf356e22bb0d6a74b0b9bc29b62c3534b`, was replaced so that W's CPU is read as cycles and W's two churn cells can never be resolved. No pilot session had run then.

Three commits of the analysis code bear on the results. The simulation that sized R_C ran at `9de26d614c85b606fb5169ed16528b7189a87ac4`. The decisions were computed from a clone at `7c1310adc4b3029d1b377d1d47b756f494916615`, whose directory of analysis code is byte for byte the sizing commit's, as their git trees show. The macros were written at `204555104f0191152d1f4bf392d2a76b0aebc1b5`, the escaping's commit. It differs from the sizing commit in the macro writer and its test alone. The analysis ran on W with numpy 2.5.0 on Python 3.14.5, twice, and the two runs were byte-identical.

The frozen text remarks that the jackknife acceleration of a median is zero at an even R. Beyond floating-point noise it is not zero in M2's HTTP/1.1 cell

on epoll, nor in some intervals of WL4. In that cell the two middle values differ by the smallest step of a floating-point number, and it passes either way.

6.7. Outputs Produced after the Analysis

The frozen text lists four descriptive outputs that the analysis code does not compute. They are B2's distributions of lateness and decision latency, the operations and copies per connection, the system calls of the proxies, and the check of the server's system-call counters. The first draft of these results reported none of them. A reading in the revision log says how each is produced. It was made after the confirmatory analysis, before any generator wrote their values and before the untimed job below ran. Generators outside the analysis code write them, each input named by its SHA-256.

Two come from archived rows. B2's distributions come from the hard-case runs, and the operations and copies of in-process dispatch from the counters of the measured windows. The rest needed untimed windows that no job had run on the frozen build. They are the proxies' system calls, the counter check, and the relay in peek, which no measured window runs, beside the relay in replay under the same load. They ran as one lab job, st1, on L after the analysis, with the frozen build and the frozen tool written for these windows. That tool predates the freeze guard and calls none. A wrapper outside the repository's tree, kept in the job's archive, ran the guard first. The job's clearance names the code freeze and the later change of Section 6.2. The tool also marks every row as development data, a constant it has carried since engineering. The reading records that this field does not make the job's rows development data, since they come from the frozen build after the pilot entry. A few descriptions were added after the values were seen: B2's runs of HC21 and HC22 counted apart, and the largest values over some tables. They describe the reported values and are listed in the revision log. None of the four decides anything, and no decision moved.

6.8. The Kernel Census

The confirmatory runs did not ask the kernel for this count. On L, `ss` was used for established sockets, for sockets in TIME-WAIT, and in diagnostics of a stalled proxy. On W, the TCP table was read for TIME-WAIT entries. Neither reading is a count of listening descriptors. The server's listening lines are its own report. The census of Section 3.3 was read on 22 October 2026 from the measured Release binaries, with no load. The revision log records it, and it changes no cost decision.

6.9. Reproducibility

The code freeze is the commit `ff2679cc89d23e08a817308c9aeba7901e056fef`. Every measured build compiled inputs that green sanitizer records of that commit cover, on the host that ran it, and every row is bound to its host's gate. The records and the gates are in the paper's repository. The raw outputs of the pilot and of every job of windows or hard cases after the pilot entry are

the assets of the repository's release `data-2026-10`. Each is archived with its SHA-256. The results files name every input they read by its SHA-256, and the generators of this paper's macros and tables check that their outputs match the committed files. The pre-specification, with its revision log, is in the paper's repository.

7. Discussion

7.1. What the Results Mean for the Claim

The question was whether one listener stays inside a ratio of 0.98 to 1.02 of a dedicated port. For this server, under load of one protocol at a time, that holds in 8 of the configurations tested. In each, churn throughput, keep-alive throughput and the median time to first byte stayed within the margin of 0.98 to 1.02 of dedicated mode. These configurations span two hosts, three backends and three protocols, HTTP/1.1, TLS and MQTT, and nothing outside them.

Equivalence within the margin is the claim; a cost of zero is not. All C1 and C3 medians on L lie on dedicated mode's side of one, and so do the server's CPU ratios of those cells. No test of a nonzero cost was planned, and the interval of C1's TLS cell on io_uring contains one.

The cost claim does not hold for h2c on L, which fails C3 on both backends. On epoll the analysis reports a measured cost, which appears as latency at a fixed load, since h2c's churn and keep-alive cells pass. On W the claim is open for h2c and MQTT: their churn measured the generator, and their open-loop cells were not resolved. Two further results bound what one listener can promise. The mixed-protocol cell on io_uring had no valid session, in both modes. That is a limitation of the server's io_uring backend, not a cost of detection, but the cost claims cannot be extended to mixed load on io_uring. And sslh-ev held a smaller counted footprint per pending connection than the server, in both cases and on both backends. So did Netty, a descriptive cell, in the partial-ClientHello case.

B1 and B2 hold where they were tested. The detector decided every hard case as the frozen table specifies, on every backend and in every mode, and no timed event broke its bounds. B3 holds against nginx, HAProxy and Envoy in both cases and hyper-util in the silent case. By B3's test each held at least 1.10 times the server's counted footprint per pending connection, as configured. The differences range from 1,837.8 to 10,726.3 bytes, with the exclusions and residuals of WL7 (Section 5.2). B3 is lost against sslh-ev. The mechanisms mostly behave as designed. Replay beat peek in 5 of 6 cells, and in-process dispatch used much less CPU than relay for HTTP/1.1. The server's relay beat every proxy it could be tested against, which excludes sslh-ev. These are results for this server's design, not for one-port listening in general.

7.2. Hypotheses for Later Work

The runs measured where the claim fails but not why. The following are hypotheses only; no run of this paper tested them.

h2c at a fixed load on L. The extra latency does not track extra work. The CPU ratio of C3's h2c cell on epoll, 1.0184, is smaller than HTTP/1.1's, 1.0308, whose C3 cell passes. On io_uring the two are about equal, 1.0184 and 1.0183. So we suspect a wait rather than work: a step on h2c's path in one-port mode that delays the server's first byte without using the CPU. A trace of each connection's events in both modes, at the C3 rate, would locate it.

The mixed cell on io_uring. The backend's ring holds 128 free buffers per worker, and the cell's background keeps as many keep-alive connections busy. We suspect that the background's receives, completing together, empty the ring before the worker gives it new buffers. The churn's receives would then fail and be posted again, and its probe would time out. The server counts such retries, and this paper reports no value of that counter. The epoll backend, which reads into each connection's own buffer, served the same cell. A larger ring is a compiled change after the code freeze, so this paper does not make it. It is planned as an exploratory follow-up, labelled as such, and the sizing of the ring is later work.

sslh-ev's memory. Both systems keep a pending connection near the same cost, so neither can reach the margin against the other. Two parts may explain sslh-ev's lead. In the partial-ClientHello case its U per pending connection, 524.3 bytes on epoll, is below the server's, 791.3. We suspect the buffer each holds for the bytes received so far. On io_uring the server's K_s exceeded K_{BASE} in most cells, while every competitor's fell below it. In the silent case $K_s - K_{\mathrm{BASE}}$ is 41.3 bytes for the server against -142.5 for sslh-ev. We suspect kernel memory that the io_uring backend holds per connection. A server that held a smaller buffer until the first decision, and a count of the slab objects per backend, could test both.

Two listeners on io_uring. A `SO_REUSEPORT` group served 1.9727 times the shared listener's connections per second on io_uring, but about the same on epoll. We suspect that the two workers' multishot accepts on one shared socket do not spread connections evenly between the workers. Counting each worker's accepts would test it. That question belongs to the accept models, outside this paper.

7.3. Threats to Validity

Scope. Every claim is about one server, as built, on two hosts, over loopback. A network between client and server adds latency and interrupt work that loopback lacks, and we expect both to dilute the cost of detection further. The cost claims hold for load of one protocol at a time. The mixed-protocol cell is secondary, and on io_uring it had no valid session. Each claim is made per protocol, backend and host, never pooled.

What the cells measure. The TLS cells of the cost family have little power to show the cost of detection, since the handshake dominates each connection. In the SSH cells the two arms exchange messages in a different order. M1 on IOCP compares replay with a peek that would switch to replay at its first undecided read, because Windows offers no low-water mark; in the runs no peek switched. On W the open-loop cells of h2c and MQTT ran at rates taken from

generator-bound churn, so they ran at a smaller share of the server's capacity than open HTTP/1.1.

Engineering before the freeze. The server was engineered until it won where it could, on development data, before the code freeze. Those runs could not time one-port mode against dedicated mode, and the revision log lists no pair of runs that would give such a ratio indirectly. The A/A pilot that sized the cost family held no one-port data. So neither the margin nor R_C could be tuned to a cost contrast. The relay and the footprint test were informed by development runs against the competitors, and a reader should weigh that history.

The footprint test. W is the footprint of each system as configured, not of its detection design alone. Two residuals of the socket-buffer accounting remain, and both sit with a system that leaves bytes queued: nginx and Envoy in the partial-ClientHello case, and the server only in peek. The co-tenants of the merged cache on L grow only when the kernel creates workqueue pools or chained scatter-gather lists. The merged cache's growth is reported per arm in Tables A.17 and A.18, and in the silent case no system leaves a byte queued. Against a system whose footprint is near a bare socket's, little room is left for the server to reach the margin. sslh-ev is such a system, as development runs before the freeze had suggested. The JVM and Go systems hold memory that their collectors manage, and their cells are descriptive.

Windows. W's windows were validated without a frequency rule, since no counter passed its test, so a change of clock speed within a W session would go undetected. W cannot count listen overflows, so a refused connection shows only as a failed connect, which invalidates its window. W's job-start check was lowered, so a job could start on a busier W than first planned; every window rule still applied.

Cells that did not run. Three outcomes were foreseen before the runs. M2's TLS cells were not run, which narrows M2 to HTTP/1.1, and sslh-ev's M3 cells are untested, which narrows M3. TLS with session resumption, a secondary cell, was not run and narrows nothing. Decided before the confirmatory runs, W's churn of h2c and MQTT ran in the pilot only, and the two-core cell on IOCP was not run. Not foreseen were the SSH cells of C2, which could not run, and the mixed cell on io_uring. Four descriptive outputs were produced after the analysis, and the system-call counts and the relay's operations came from an untimed job run then (Section 6.7).

Competitors and instruments. Each competitor runs the configuration its project documents for the compared feature, and a better one may exist. HAProxy's client, server and connect timeouts stay at its default, which is none. The runs use kernel 7.2.6-arch2-1, while the pre-specification cites the source at tag v7.2; each cited line is unchanged there, but the distribution's own patches were not compared. A closed-loop cell measures throughput at 64 connections per server core only.

Analysis. Two hosts with different Python versions computed every decision equal (Section 6.4). B1's hard cases and B2 were read from the runners' rows rather than computed by the analysis code, as a logged reading states (Section 6.3).

8. Conclusions

A server that reads a connection's first bytes and decides its protocol on one listening socket can match the same server with one port per protocol. For `oneport`, under load of one protocol at a time, one-port mode was equivalent to dedicated mode within 0.98 to 1.02 in 30 of the 32 tested cost cells. The equivalence claim holds for 8 configurations: HTTP/1.1, TLS and MQTT on both Linux backends, and HTTP/1.1 and TLS on Windows. The TLS cells have little power to show the cost of detection, since the handshake dominates. The claim does not hold for h2c on L, where equivalence of the time to first byte at a fixed load was not shown. On epoll the analysis reports a measured cost there. On W the claim is open for h2c and MQTT.

The detector decided all 26,240 runs of the frozen hard cases as specified, with no timed event out of bounds. By B3's test, nginx, HAProxy and Envoy in both cases, and hyper-util in the silent case, held at least 1.10 times the server's counted footprint per pending connection, as configured. Their medians of the difference range from 1,837.8 to 10,726.3 bytes, with the intervals and exclusions of Section 5.2. sslh-ev held less than the server in every cell, as did Netty in the partial-ClientHello case. Replay served more connections per second than peek in every cell but h2c on IOCP. In-process dispatch used less CPU than relay for HTTP/1.1. The server's relay served more connections per second than four of the five proxies, and sslh-ev could not be tested.

The runs also exposed a limit of the server's io_uring backend: beside a busy keep-alive background, its churn starved in both modes. A larger ring of provided buffers is the first follow-up, labelled exploratory, and the cause of h2c's latency at a fixed load is the second.

CRediT authorship contribution statement

Alex I. Tsvetanov is the sole author. He did the conceptualization, the methodology, the software, the validation, the formal analysis and the investigation. He also did the resources, the data curation, the writing, the visualization, the supervision and the project administration. He acquired the funding. He is responsible for the manuscript.

Funding

The publication of this paper was financially supported by the scientific research sector of the Technical University of Sofia.

Data availability

The repository `https://github.com/Alex-Tsvetanov/paper-one-port` holds the pre-specification with its revision log. It also holds the server, the generators, the competitors' configurations and the analysis. It holds the

generated macro files and the programs that write this paper's tables. The code freeze is commit `ff2679cc89d23e08a817308c9aeba7901e056fef`. The raw outputs of the pilot, and of every later job of windows or hard cases, are the assets of release `data-2026-10` (`https://github.com/Alex-Tsvetanov/paper-one-port/releases/tag/data-2026-10`). Each asset has its SHA-256 beside it. `lab/archives.sha256` lists them. The revision log describes the development runs before the pilot entry. Their raw outputs are not published. The repository's `lab/` directory holds the sanitizer records of the measured builds and the two hosts' gates. It also holds the simulation behind the pilot entry, and rule E's file, which the analysis read. Each record names the SHA-256 of an archive of its logs. Those logs are not published. [For Alex: the DOIs of the repository and of release `data-2026-10`, which need Alex's account (for example a Zenodo archive of the release).]

Acknowledgements

The author thanks the scientific research sector of the Technical University of Sofia for support for publishing this paper.

Declaration of competing interest

The author declares no competing interest. The funder had no role in the study, in the analysis, or in the decision to publish.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work a language model in Cursor was used to retarget the manuscript to Computer Communications and to replace one cost sentence with the measured interval. The measured numbers were not changed. [For Alex: confirm this declaration, including the tool name, before submission. The decision of 8 October 2026 was that the earlier draft would carry no such statement.]

Appendix A. Per-Cell Tables

This appendix holds the per-cell tables that Sections 5 and 7 cite. Each is generated from the results files, and every value in it is a macro of the generated macro files or a count from the results files.

Table A.12: Every invalid window of a family cell, per arm, with its reasons. The count after a reason is the number of windows that gave it; a window may give several.

Cell	Arm	Windows	Reasons
C3.W.IOCP.http1	one-port	1	connects failed (1)
C3.W.IOCP.h2c	dedicated	1	connects failed (1); error share above its bound (1); too few of the exchanges due completed (1)
C3.W.IOCP.mqtt	one-port	1	the job was stopped during the window (1)
B3.silent.nginx.epoll	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.nginx.epoll	server	2	TIME-WAIT count at a sample differed from the baseline (2)
B3.silent.haproxy.epoll	server	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.haproxy.io_uring	competitor	2	TIME-WAIT count at a sample differed from the baseline (2)
B3.silent.haproxy.io_uring	server	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.envoy.epoll	server	2	TIME-WAIT count at a sample differed from the baseline (2)
B3.silent.envoy.io_uring	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.envoy.io_uring	server	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.sslh-ev.epoll	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.sslh-ev.io_uring	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.hyper-util.epoll	server	2	TIME-WAIT count at a sample differed from the baseline (2)
B3.partial-hello.nginx.epoll	competitor	2	TIME-WAIT count at a sample differed from the baseline (2)
B3.partial-hello.nginx.io_uring	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.partial-hello.haproxy.epoll	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.partial-hello.haproxy.epoll	server	2	TIME-WAIT count at a sample differed from the baseline (2)
B3.partial-hello.envoy.io_uring	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.partial-hello.sslh-ev.epoll	competitor	2	TIME-WAIT count at a sample differed from the baseline (1); W moved beyond its settling bound (1)
B3.partial-hello.sslh-ev.epoll	server	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.partial-hello.sslh-ev.io_uring	competitor	2	TIME-WAIT count at a sample differed from the baseline (2)
B3.partial-hello.sslh-ev.io_uring	server	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.netty.io_uring	server	2	TIME-WAIT count at a sample differed from the baseline (2)
B3.silent.jetty.epoll	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.jetty.epoll	server	2	TIME-WAIT count at a sample differed from the baseline (2)
B3.silent.caddy-l4.io_uring	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.cmux.epoll	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.silent.cmux.epoll	server	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.partial-hello.netty.epoll	competitor	2	TIME-WAIT count at a sample differed from the baseline (2)
B3.partial-hello.netty.epoll	server	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.partial-hello.jetty.io_uring	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.partial-hello.caddy-l4.epoll	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.partial-hello.caddy-l4.io_uring	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.partial-hello.cmux.epoll	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
B3.partial-hello.cmux.epoll	server	4	TIME-WAIT count at a sample differed from the baseline (4)
B3.partial-hello.cmux.io_uring	competitor	1	TIME-WAIT count at a sample differed from the baseline (1)
M3.L.epoll.tls-stub.sslh-ev	proxy	40	error share above its bound (40); connects failed (6)

Table A.13: The secondary metrics of the cost cells, one-port / dedicated. WL4 gives the server's CPU per exchange and its resident memory at the end and at its peak. WL2 gives the time to first byte at the 99th percentile, in C3 only. Each is the median with its 95% interval. On W the CPU measure is the server's processor cycles, read with `QueryProcessCycleTime`; a ratio of two arms needs no conversion.

Hyp.	Host, backend	Protocol	CPU	Memory, end	Memory, peak	TTFB, tail
C1	L, epoll	HTTP/1.1	1.0137 [1.0130, 1.0147]	1.0005 [0.9959, 1.0090]	1.0005 [0.9959, 1.0090]	none
C1	L, epoll	h2c	1.0056 [1.0043, 1.0090]	0.9133 [0.9050, 0.9221]	0.9133 [0.9050, 0.9221]	none
C1	L, epoll	TLS	1.0019 [1.0005, 1.0026]	1.0000 [0.9955, 1.0032]	1.0004 [0.9969, 1.0037]	none
C1	L, epoll	MQTT	1.0116 [1.0107, 1.0124]	1.0002 [0.9956, 1.0049]	1.0002 [0.9956, 1.0049]	none
C1	L, io_uring	HTTP/1.1	1.0102 [1.0072, 1.0112]	0.9998 [0.9944, 1.0048]	0.9998 [0.9944, 1.0048]	none
C1	L, io_uring	h2c	1.0242 [1.0175, 1.0301]	0.9866 [0.9772, 0.9960]	0.9902 [0.9845, 0.9964]	none
C1	L, io_uring	TLS	1.0014 [0.9995, 1.0039]	1.0059 [0.9851, 1.0275]	1.0081 [0.9845, 1.0280]	none
C1	L, io_uring	MQTT	1.0085 [1.0075, 1.0108]	1.0013 [0.9918, 1.0050]	1.0013 [0.9918, 1.0050]	none
C1	W, IOCP	HTTP/1.1	1.0045 [1.0005, 1.0090]	0.9782 [0.9778, 0.9786]	0.9782 [0.9779, 0.9786]	none
C1	W, IOCP	h2c	none	none	none	none
C1	W, IOCP	TLS	0.9999 [0.9955, 1.0054]	0.9911 [0.9685, 1.0086]	0.9926 [0.9900, 0.9983]	none
C1	W, IOCP	MQTT	none	none	none	none
C2	L, epoll	HTTP/1.1	1.0011 [0.9990, 1.0026]	0.9945 [0.9919, 1.0021]	0.9945 [0.9919, 1.0021]	none
C2	L, epoll	h2c	0.9991 [0.9962, 1.0025]	1.0003 [0.9961, 1.0088]	1.0003 [0.9961, 1.0088]	none
C2	L, epoll	TLS	1.0029 [0.9979, 1.0045]	0.9993 [0.9976, 1.0001]	0.9993 [0.9974, 1.0001]	none
C2	L, epoll	MQTT	1.0011 [0.9996, 1.0028]	1.0010 [0.9956, 1.0057]	1.0010 [0.9956, 1.0057]	none
C2	L, io_uring	HTTP/1.1	1.0010 [0.9996, 1.0028]	1.0003 [0.9955, 1.0045]	1.0003 [0.9954, 1.0045]	none
C2	L, io_uring	h2c	1.0002 [0.9986, 1.0032]	1.0008 [0.9994, 1.0034]	1.0008 [0.9994, 1.0034]	none
C2	L, io_uring	TLS	0.9997 [0.9958, 1.0026]	0.9986 [0.9939, 1.0026]	0.9986 [0.9939, 1.0026]	none
C2	L, io_uring	MQTT	1.0006 [0.9997, 1.0027]	1.0004 [0.9937, 1.0047]	1.0004 [0.9937, 1.0050]	none
C2	W, IOCP	HTTP/1.1	0.9967 [0.9884, 0.9995]	0.9789 [0.9787, 0.9792]	0.9789 [0.9787, 0.9792]	none
C2	W, IOCP	h2c	0.9984 [0.9918, 1.0038]	0.9779 [0.9777, 0.9781]	0.9779 [0.9777, 0.9781]	none
C2	W, IOCP	TLS	0.9983 [0.9921, 1.0043]	0.9754 [0.9736, 0.9805]	0.9870 [0.9865, 0.9878]	none
C2	W, IOCP	MQTT	1.0011 [0.9987, 1.0050]	0.9792 [0.9791, 0.9793]	0.9792 [0.9791, 0.9793]	none
C3	L, epoll	HTTP/1.1	1.0308 [1.0291, 1.0325]	0.9978 [0.9914, 1.0060]	0.9978 [0.9914, 1.0060]	1.0239 [0.9911, 1.0443]
C3	L, epoll	h2c	1.0184 [1.0151, 1.0202]	1.0018 [0.9838, 1.0124]	1.0018 [0.9838, 1.0123]	1.0255 [1.0222, 1.0446]

Table A.14: The analysis clustered by lab job, which decides nothing: the cost cells that ran in more than one job, all on W. Beside each clustered interval is the cell's interval by session.

Hyp.	Host, backend	Protocol	Median	Interval by job	Interval by session
C1	W, IOCP	HTTP/1.1	0.9978	[0.9951, 0.9993]	[0.9951, 0.9995]
C1	W, IOCP	TLS	0.9969	[0.9867, 0.9979]	[0.9928, 1.0039]
C2	W, IOCP	HTTP/1.1	1.0024	[1.0023, 1.0024]	[0.9985, 1.0142]
C2	W, IOCP	h2c	1.0002	[0.9994, 1.0050]	[0.9968, 1.0071]
C2	W, IOCP	TLS	1.0003	[0.9948, 1.0009]	[0.9959, 1.0108]
C2	W, IOCP	MQTT	0.9977	[0.9968, 1.0002]	[0.9932, 1.0014]
C3	W, IOCP	HTTP/1.1	1.0030	[0.9986, 1.0037]	[0.9983, 1.0045]
C3	W, IOCP	h2c	0.9906	[0.9687, 0.9995]	[0.9582, 1.0131]
C3	W, IOCP	TLS	0.9988	[0.9982, 1.0028]	[0.9957, 1.0021]
C3	W, IOCP	MQTT	1.0183	[1.0151, 1.0219]	[0.9889, 1.0360]

Table A.15: The secondary cells with sessions of their own, each with its valid sessions, median and 95% interval, or whether it was not run or has no valid session. Section 5.4 gives the reasons.

Cell	Host, backend	Ratio, metric	Valid	Median	95% interval
SSH, C1	L, epoll	one-port / dedicated, conn./s	16	0.9863	[0.9840, 0.9876]
SSH, C1	L, io_uring	one-port / dedicated, conn./s	16	0.9849	[0.9835, 0.9860]
SSH, C1	W, IOCP	one-port / dedicated, conn./s	16	1.0077	[0.9994, 1.0128]
SSH, C2	L, epoll	one-port / dedicated, req./s	0	no valid session	
SSH, C2	L, io_uring	one-port / dedicated, req./s	0	no valid session	
SSH, C2	W, IOCP	one-port / dedicated, req./s	0	no valid session	
SSH, C3	L, epoll	one-port / dedicated, median TTFB	16	1.1105	[1.1091, 1.1117]
SSH, C3	L, io_uring	one-port / dedicated, median TTFB	16	1.0642	[1.0588, 1.0679]
SSH, C3	W, IOCP	one-port / dedicated, median TTFB	16	1.1036	[1.0974, 1.1058]
mixed protocols, C1	L, epoll	one-port / dedicated, conn./s	16	0.9958	[0.9938, 0.9971]
mixed protocols, C1	L, io_uring	one-port / dedicated, conn./s	0	no valid session	
mixed protocols, C1	W, IOCP	one-port / dedicated, conn./s	16	0.9939	[0.9871, 0.9995]
TLS, resumption	L, epoll	one-port / dedicated, conn./s	0	not run	
TLS, ALPN h2	L, epoll	one-port / dedicated, conn./s	16	0.9957	[0.9946, 0.9991]
TLS, resumption	L, io_uring	one-port / dedi-	0	not run	

Table A.16: The secondary cells of the mechanisms and of B3. They give the time to first byte at a fixed load for M1 and M3 and the CPU per connection of every mechanism cell. For M2 and M3 that CPU is the front's. The last rows give B3 with the server in its other detection mode.

Cell	Host, backend	Ratio metric	V	Valid	Median	95% interval
M1 TTFB, HTTP/1.1	L, epoll	default / other, median TTFB	16		0.9876	[0.9865, 0.9887]
M1 CPU, HTTP/1.1	L, epoll	default / other, CPU per conn.	16		0.9548	[0.9524, 0.9554]
M1 TTFB, h2c	L, epoll	default / other, median TTFB	16		0.9812	[0.9796, 0.9839]
M1 CPU, h2c	L, epoll	default / other, CPU per conn.	16		0.9594	[0.9551, 0.9622]
M1 TTFB, HTTP/1.1	L, io_uring	default / other, median TTFB	16		0.9956	[0.9949, 0.9967]
M1 CPU, HTTP/1.1	L, io_uring	default / other, CPU per conn.	16		0.9148	[0.9128, 0.9158]
M1 TTFB, h2c	L, io_uring	default / other, median TTFB	16		0.9751	[0.9742, 0.9759]
M1 CPU, h2c	L, io_uring	default / other, CPU per conn.	16		0.9416	[0.9382, 0.9451]
M1 TTFB, HTTP/1.1	W, IOCP	default / other, median TTFB	16		0.9845	[0.9833, 0.9865]
M1 CPU, HTTP/1.1	W, IOCP	default / other, CPU per conn.	16		0.9624	[0.9581, 0.9677]
M1 TTFB, h2c	W, IOCP	default / other, median TTFB	16		0.9875	[0.9801, 1.0152]
M1 CPU, h2c	W, IOCP	default / other, CPU per conn.	16		0.9655	[0.9578, 0.9739]
M2 CPU, HTTP/1.1	L, epoll	in-process	16		0.3649	[0.3635, 0.3651]

Table A.17: The parts of the competitor's footprint in each B3 cell, as medians over the valid sessions. W, U, K_q and $K_s - K_{\mathrm{BASE}}$ are in bytes per pending connection. "Skb caches" is the growth of the three socket-buffer caches subtracted from K_s. "Shared cache" is the part of it in the merged cache, whose co-tenants are `pool_workqueue` and `sgpool-16`. Both are in bytes over the host. f is the mean of the `f` field of `ss -tm` over the accepted sockets, in bytes, reported and not added.

Case	System	Backend	W	U	K_q	$K_s - K_{\mathrm{BASE}}$	Skb caches	Shared cache	f
silent	nginx	epoll	11,156.5	4,875.1	0.0	−182.5	0.0	0.0	0.0
silent	nginx	io_uring	11,189.7	4,875.1	0.0	−149.3	0.0	0.0	0.0
silent	HAProxy	epoll	9,407.2	3,087.6	0.0	−144.3	0.0	0.0	0.0
silent	HAProxy	io_uring	9,400.8	3,087.6	0.0	−150.6	0.0	0.0	0.0
silent	Envoy	epoll	16,360.4	10,089.2	0.0	−191.8	0.0	0.0	0.0
silent	Envoy	io_uring	16,442.8	10,091.2	0.0	−113.7	0.0	0.0	0.0
silent	sslh-ev	epoll	6,693.1	400.2	0.0	−171.0	0.0	0.0	0.0
silent	sslh-ev	io_uring	6,721.5	400.2	0.0	−142.5	0.0	0.0	0.0
silent	hyper-util	epoll	8,715.8	2,432.2	0.0	−180.3	0.0	0.0	0.0
silent	hyper-util	io_uring	8,709.3	2,432.2	0.0	−186.8	0.0	0.0	0.0
partial ClientHello	nginx	epoll	12,638.4	5,307.0	1,068.0	−200.5	8,065,024	17,344	4,028.0
partial ClientHello	nginx	io_uring	12,678.8	5,307.2	1,068.0	−160.3	7,798,784	17,344	4,028.0
partial ClientHello	HAProxy	epoll	13,628.5	7,308.1	0.0	−144.4	0.0	0.0	0.0
partial ClientHello	HAProxy	io_uring	13,621.6	7,308.2	0.0	−148.5	0.0	0.0	0.0
partial ClientHello	Envoy	epoll	17,868.7	10,478.2	1,068.0	−142.3	7,987,200	21,440	4,028.0
partial ClientHello	Envoy	io_uring	17,890.0	10,477.2	1,068.0	−117.5	7,897,088	5,168	4,028.0
partial ClientHello	sslh-ev	epoll	6,842.3	524.3	0.0	−145.9	0.0	0.0	0.0
partial ClientHello	sslh-ev	io_uring	6,811.2	524.3	0.0	−176.9	0.0	0.0	0.0
silent	Netty	epoll	8,074.1	1,720.8	0.0	−113.0	0.0	0.0	0.0
silent	Netty	io_uring	8,091.5	1,721.1	0.0	−94.1	0.0	0.0	0.0
silent	Jetty	epoll	11,973.2	5,715.6	0.0	−172.4	0.0	0.0	0.0
silent	Jetty	io_uring	12,077.9	5,791.8	0.0	−198.7	0.0	0.0	0.0
silent	caddy-l4	epoll	17,289.0	10,978.7	0.0	−166.5	−65,536	0.0	0.0
silent	caddy-l4	io_uring	17,264.1	10,986.3	0.0	−183.8	−120,832	0.0	0.0
silent	cmux	epoll	9,973.0	3,720.9	0.0	−211.9	−118,784	0.0	0.0
silent	cmux	io_uring	9,994.5	3,720.8	0.0	−190.2	−96,256	0.0	0.0
partial ClientHello	Netty	epoll	8,699.2	2,344.7	0.0	−154.1	0.0	0.0	0.0
partial ClientHello	Netty	io_uring	8,679.6	2,342.0	0.0	−146.3	0.0	0.0	0.0
partial ClientHello	Jetty	epoll	38,300.3	32,051.6	0.0	−177.3	0.0	0.0	0.0
partial ClientHello	Jetty	io_uring	38,684.8	32,370.9	0.0	−158.5	0.0	0.0	0.0
partial ClientHello	caddy-l4	epoll	20,237.6	13,919.9	0.0	−206.3	−157,696	0.0	0.0
partial ClientHello	caddy-l4	io_uring	20,191.8	13,895.6	0.0	−169.0	−196,608	0.0	0.0
partial ClientHello	cmux	epoll	18,826.2	12,533.6	0.0	−175.7	−147,456	0.0	0.0
partial ClientHello	cmux	io_uring	18,825.8	12,533.0	0.0	−170.9	−104,448	0.0	0.0

Table A.18: The parts of the server's footprint in each B3 cell, in the units of Table A.17.

Case	System	Backend	W	U	K_q	$K_s - K_{\mathrm{BASE}}$	Skb caches	Shared cache	f
silent	nginx	epoll	6,677.2	407.1	0.0	−193.8	0.0	0.0	0.0
silent	nginx	io_uring	6,883.5	407.1	0.0	12.5	0.0	0.0	0.0
silent	HAProxy	epoll	6,737.1	407.1	0.0	−133.9	0.0	0.0	0.0
silent	HAProxy	io_uring	6,885.5	407.1	0.0	14.4	0.0	0.0	0.0
silent	Envoy	epoll	6,673.5	407.1	0.0	−197.5	0.0	0.0	0.0
silent	Envoy	io_uring	6,943.2	407.1	0.0	72.2	0.0	0.0	0.0
silent	sslh-ev	epoll	6,708.3	407.1	0.0	−162.7	0.0	0.0	0.0
silent	sslh-ev	io_uring	6,912.3	407.1	0.0	41.3	0.0	0.0	0.0
silent	hyper-util	epoll	6,692.9	402.2	0.0	−173.3	0.0	0.0	0.0
silent	hyper-util	io_uring	6,885.6	402.2	0.0	19.5	0.0	0.0	0.0
partial ClientHello	nginx	epoll	7,055.3	791.3	0.0	−200.0	−4,096.0	0.0	0.0
partial ClientHello	nginx	io_uring	7,274.5	791.3	0.0	19.3	−4,096.0	0.0	0.0
partial ClientHello	HAProxy	epoll	7,119.1	791.3	0.0	−136.2	0.0	0.0	0.0
partial ClientHello	HAProxy	io_uring	7,294.4	791.3	0.0	39.1	0.0	0.0	0.0
partial ClientHello	Envoy	epoll	7,137.1	791.3	0.0	−118.2	0.0	0.0	0.0
partial ClientHello	Envoy	io_uring	7,374.4	791.3	0.0	119.2	−8,192.0	0.0	0.0
partial ClientHello	sslh-ev	epoll	7,145.0	791.3	0.0	−110.3	0.0	0.0	0.0
partial ClientHello	sslh-ev	io_uring	7,247.3	791.3	0.0	−8.0	0.0	0.0	0.0
silent	Netty	epoll	6,785.6	402.2	0.0	−80.5	0.0	0.0	0.0
silent	Netty	io_uring	6,954.3	402.2	0.0	88.2	0.0	0.0	0.0
silent	Jetty	epoll	6,683.5	402.2	0.0	−182.6	0.0	0.0	0.0
silent	Jetty	io_uring	6,740.1	402.2	0.0	−126.1	0.0	0.0	0.0
silent	caddy-l4	epoll	6,731.4	407.1	0.0	−139.7	0.0	0.0	0.0
silent	caddy-l4	io_uring	6,827.0	407.1	0.0	−44.0	0.0	0.0	0.0
silent	cmux	epoll	6,677.7	402.2	0.0	−188.4	0.0	0.0	0.0
silent	cmux	io_uring	6,835.7	402.2	0.0	−30.4	0.0	0.0	0.0
partial ClientHello	Netty	epoll	10,853.4	4,521.2	0.0	−131.7	0.0	0.0	0.0
partial ClientHello	Netty	io_uring	11,047.0	4,522.8	0.0	60.3	0.0	0.0	0.0
partial ClientHello	Jetty	epoll	10,720.1	4,521.2	0.0	−265.0	0.0	0.0	0.0
partial ClientHello	Jetty	io_uring	10,942.2	4,522.8	0.0	−44.5	0.0	0.0	0.0
partial ClientHello	caddy-l4	epoll	7,087.2	791.3	0.0	−168.0	0.0	0.0	0.0
partial ClientHello	caddy-l4	io_uring	7,273.6	791.3	0.0	18.3	0.0	0.0	0.0
partial ClientHello	cmux	epoll	10,836.4	4,521.2	0.0	−148.7	0.0	0.0	0.0
partial ClientHello	cmux	io_uring	10,991.4	4,522.8	0.0	4.7	0.0	0.0	0.0

Table A.19: Operations and copies of the cost cells, per connection in C1 and C3 and per request in C2, from the valid windows (WL5). Each value is the mean over an arm's valid windows; "Windows" gives them as one-port/dedicated. Operations are the system calls and ring submissions the counters name. The bytes are payload bytes, copied in user space by the server's own code or received and sent across the boundary. Both arms run replay and in-process dispatch. No byte was peeked in any arm.

				Operations		Copied		Received		Sent	
Hyp.	**Host, backend**	**Protocol**	**Windows**	**One-port**	**Ded.**	**One-port**	**Ded.**	**One-port**	**Ded.**	**One-port**	**Ded.**
C1	L, epoll	HTTP/1.1	56/56	4.048	4.048	0.0	0.0	57.0	57.0	78.0	78.0
C1	L, epoll	h2c	56/56	6.047	6.047	0.0	0.0	76.0	76.0	70.0	70.0
C1	L, epoll	TLS	56/56	6.080	6.081	0.0	0.0	354.0	354.0	913.0	913.0
C1	L, epoll	MQTT	56/56	6.047	6.048	0.0	0.0	16.0	16.0	4.0	4.0
C1	L, io_uring	HTTP/1.1	56/56	4.032	4.032	0.0	0.0	57.0	57.0	78.0	78.0
C1	L, io_uring	h2c	56/56	6.032	6.032	0.0	0.0	76.0	76.0	70.0	70.0
C1	L, io_uring	TLS	56/56	7.051	7.051	0.0	0.0	354.0	354.0	913.0	913.0
C1	L, io_uring	MQTT	56/56	6.032	6.032	0.0	0.0	16.0	16.0	4.0	4.0
C1	W, IOCP	HTTP/1.1	56/56	4.342	4.345	0.0	0.0	57.0	57.0	78.0	78.0
C1	W, IOCP	TLS	56/56	7.040	7.059	0.0	0.0	354.0	354.0	913.0	913.0
C2	L, epoll	HTTP/1.1	56/56	2.016	2.016	0.0	0.0	38.0	38.0	78.0	78.0
C2	L, epoll	h2c	56/56	2.016	2.016	0.0	0.0	26.0	26.0	38.0	38.0
C2	L, epoll	TLS	56/56	2.017	2.017	0.0	0.0	60.0	60.0	100.1	100.1
C2	L, epoll	MQTT	56/56	2.017	2.017	0.0	0.0	2.0	2.0	2.0	2.0
C2	L, io_uring	HTTP/1.1	56/56	3.016	3.016	0.0	0.0	38.0	38.0	78.0	78.0
C2	L, io_uring	h2c	56/56	3.016	3.016	0.0	0.0	26.0	26.0	38.0	38.0
C2	L, io_uring	TLS	56/56	3.016	3.016	0.0	0.0	60.0	60.0	100.1	100.1
C2	L, io_uring	MQTT	56/56	3.016	3.016	0.0	0.0	2.0	2.0	2.0	2.0
C2	W, IOCP	HTTP/1.1	56/56	3.017	3.017	0.0	0.0	38.0	38.0	78.0	78.0
C2	W, IOCP	h2c	56/56	3.017	3.017	0.0	0.0	26.0	26.0	38.0	38.0
C2	W, IOCP	TLS	56/56	3.017	3.018	0.0	0.0	60.0	60.0	100.1	100.1
C2	W, IOCP	MQTT	56/56	3.017	3.017	0.0	0.0	2.0	2.0	2.0	2.0
C3	L, epoll	HTTP/1.1	56/56	6.941	6.944	0.0	0.0	57.0	57.0	78.0	78.0
C3	L, epoll	h2c	56/56	8.974	8.975	0.0	0.0	76.0	76.0	70.0	70.0
C3	L, epoll	TLS	56/56	10.955	11.032	0.0	0.0	354.0	354.0	913.0	913.0
C3	L, epoll	MQTT	56/56	8.965	8.962	0.0	0.0	16.0	16.0	4.0	4.0
C3	L, io_uring	HTTP/1.1	56/56	5.993	5.999	0.0	0.0	57.0	57.0	78.0	78.0
C3	L, io_uring	h2c	56/56	9.014	9.019	0.0	0.0	76.0	76.0	70.0	70.0
C3	L, io_uring	TLS	56/56	11.001	11.003	0.0	0.0	354.0	354.0	913.0	913.0
C3	L, io_uring	MQTT	56/56	9.005	9.006	0.0	0.0	16.0	16.0	4.0	4.0
C3	W, IOCP	HTTP/1.1	57/58	5.936	5.943	0.0	0.0	57.0	57.0	78.0	78.0
C3	W, IOCP	h2c	58/57	8.283	8.317	0.0	0.0	76.0	76.0	70.0	70.0
C3	W, IOCP	TLS	56/56	11.693	11.708	0.0	0.0	354.0	354.0	913.0	913.0
C3	W, IOCP	MQTT	56/56	8.379	8.379	0.0	0.0	16.0	16.0	4.0	4.0

Table A.20: Operations and copies per connection of M1's cells, one-port mode with in-process dispatch in each detection mode, as means over the valid windows, in the units of Table A.19. "Wakeups" counts the wakeups during detection.

Host, backend	**Protocol**	**Mode**	**Windows**	**Operations**	**Copied**	**Received**	**Sent**	**Peeked**	**Wakeups**
L, epoll	HTTP/1.1	replay	32	4.048	0.0	57.0	78.0	0.0	1.000
L, epoll	HTTP/1.1	peek	32	5.048	0.0	57.0	78.0	24.0	1.000
L, epoll	h2c	replay	32	6.047	0.0	76.0	70.0	0.0	1.000
L, epoll	h2c	peek	32	7.047	0.0	76.0	70.0	24.0	1.000
L, io_uring	HTTP/1.1	replay	32	4.032	0.0	57.0	78.0	0.0	1.000
L, io_uring	HTTP/1.1	peek	32	6.048	0.0	57.0	78.0	24.0	1.000
L, io_uring	h2c	replay	32	6.032	0.0	76.0	70.0	0.0	1.000
L, io_uring	h2c	peek	32	8.047	0.0	76.0	70.0	24.0	1.000
W, IOCP	HTTP/1.1	replay	32	4.019	0.0	57.0	78.0	0.0	1.000
W, IOCP	HTTP/1.1	peek	32	6.019	0.0	57.0	78.0	24.0	1.000
W, IOCP	h2c	replay	32	7.974	0.0	76.0	70.0	0.0	1.000
W, IOCP	h2c	peek	32	9.867	0.0	76.0	70.0	24.0	1.000

Table A.21: Operations and copies per relayed connection of the server's relay on L, from st1's untimed windows, in front of the stub as in M3. The load is open-loop churn at 2,000 exchanges per second. Each value is the front's count over its process divided by the connections it relayed, in the units of Table A.19. The loop's waits and the accepts that find no connection depend on the load.

Backend	Protocol	Mode	Relayed	Operations	Copied	Received	Sent	Peeked	Wakeups
epoll	TLS (SNI)	replay	5,002	17.860	0.0	224.0	224.0	0.0	1.000
epoll	TLS (SNI)	peek	5,002	20.341	0.0	224.0	224.0	235.0	1.000
epoll	HTTP/1.1	replay	5,002	17.712	0.0	135.0	135.0	0.0	1.000
epoll	HTTP/1.1	peek	5,002	19.301	0.0	135.0	135.0	24.0	1.000
io_uring	TLS (SNI)	replay	5,002	21.000	0.0	224.0	224.0	0.0	1.000
io_uring	TLS (SNI)	peek	5,002	25.001	0.0	224.0	224.0	235.0	1.000
io_uring	HTTP/1.1	replay	5,002	21.000	0.0	135.0	135.0	0.0	1.000
io_uring	HTTP/1.1	peek	5,002	24.001	0.0	135.0	135.0	24.0	1.000

Table A.22: The check of the server's system-call counters, one untimed window per cost cell of L in each mode, from st1. Each entry gives the row's checks that equal the count of the same calls, by `perf stat` (the check) and by `perf trace -s` (its cross-check), over the row's checks. The shortfall is `perf trace -s`'s largest, as a percentage of `perf stat`'s count of a call, over the cell's two rows. C1 and C3 share one untimed load, open-loop churn, and C2 runs keep-alive.

			Dedicated		One-port		
Hyp.	Backend	Protocol	Stat	Trace	Stat	Trace	Shortfall %
C1	epoll	HTTP/1.1	9/9	8/9	9/9	7/9	0.10
C1	epoll	h2c	9/9	8/9	9/9	6/9	0.09
C1	epoll	TLS	9/9	9/9	9/9	9/9	0.00
C1	epoll	MQTT	9/9	8/9	9/9	8/9	0.04
C1	io_uring	HTTP/1.1	6/6	6/6	6/6	6/6	0.00
C1	io_uring	h2c	6/6	5/6	6/6	5/6	0.42
C1	io_uring	TLS	6/6	6/6	6/6	6/6	0.00
C1	io_uring	MQTT	6/6	6/6	6/6	6/6	0.00
C2	epoll	HTTP/1.1	9/9	6/9	9/9	6/9	0.10
C2	epoll	h2c	9/9	6/9	9/9	6/9	0.12
C2	epoll	TLS	9/9	6/9	9/9	6/9	0.13
C2	epoll	MQTT	9/9	6/9	9/9	6/9	0.21
C2	io_uring	HTTP/1.1	6/6	4/6	6/6	4/6	0.12
C2	io_uring	h2c	6/6	4/6	6/6	4/6	0.08
C2	io_uring	TLS	6/6	4/6	6/6	4/6	0.07
C2	io_uring	MQTT	6/6	4/6	6/6	4/6	0.14
C3	epoll	HTTP/1.1	9/9	9/9	9/9	9/9	0.00
C3	epoll	h2c	9/9	6/9	9/9	6/9	0.26
C3	epoll	TLS	9/9	8/9	9/9	9/9	0.01
C3	epoll	MQTT	9/9	8/9	9/9	7/9	0.08
C3	io_uring	HTTP/1.1	6/6	6/6	6/6	6/6	0.00
C3	io_uring	h2c	6/6	5/6	6/6	5/6	0.22
C3	io_uring	TLS	6/6	6/6	6/6	6/6	0.00
C3	io_uring	MQTT	6/6	6/6	6/6	6/6	0.00

References

[1] nginx contributors, nginx, version 1.30.5, September 2026. URL `https://nginx.org/download/nginx-1.30.5.tar.gz`

[2] HAProxy contributors, HAProxy, version 3.4.6, September 2026. URL `https://www.haproxy.org/download/3.4/src/haproxy-3.4.6.tar.gz`

[3] Envoy contributors, Envoy proxy, release v1.39.2, October 2026.
URL https://github.com/envoyproxy/envoy/releases/tag/v1.39.2

[4] caddy-l4 contributors, caddy-l4, the layer 4 app for Caddy, release v0.1.2, commit 42db5690dea199f930a6f08005fe2e4aab10dcc9, July 2026.
URL https://github.com/mholt/caddy-l4/releases/tag/v0.1.2

[5] sslh contributors, sslh, release v2.3.1, March 2026.
URL https://github.com/yrutschle/sslh/releases/tag/v2.3.1

[6] Netty contributors, Netty, release 4.2.18.Final, September 2026.
URL https://github.com/netty/netty/releases/tag/netty-4.2.18.Final

[7] Jetty contributors, Eclipse Jetty, release 12.1.13, September 2026.
URL https://github.com/jetty/jetty.project/releases/tag/jetty-12.1.13

[8] cmux contributors, cmux: Connection multiplexer for Go, release v0.1.5, 2021.
URL https://github.com/soheilhy/cmux/releases/tag/v0.1.5

[9] hyper contributors, hyper-util, release v0.1.21, September 2026.
URL https://github.com/hyperium/hyper-util/releases/tag/v0.1.21

[10] M. Thomson, C. Benfield (Eds.), HTTP/2, no. 9113 in RFC, RFC Editor, 2022. doi:10.17487/RFC9113.

[11] E. Rescorla, The transport layer security (TLS) protocol version 1.3, RFC 8446, RFC Editor (August 2018). doi:10.17487/RFC8446.

[12] W. Tarreau, The PROXY protocol versions 1 & 2, hAProxy Technologies; the text shipped with HAProxy 3.4.6, dated April 2026.
URL https://www.haproxy.org/download/3.4/doc/proxy-protocol.txt

[13] R. Fielding, M. Nottingham, J. Reschke (Eds.), HTTP/1.1, no. 9112 in RFC, RFC Editor, 2022. doi:10.17487/RFC9112.

[14] R. Fielding, M. Nottingham, J. Reschke (Eds.), HTTP Semantics, no. 9110 in RFC, RFC Editor, 2022. doi:10.17487/RFC9110.

[15] L. Dusseault, J. Snell, PATCH method for HTTP, RFC 5789, RFC Editor (March 2010). doi:10.17487/RFC5789.

[16] T. Ylonen, The secure shell (SSH) transport layer protocol, RFC 4253, RFC Editor (January 2006). doi:10.17487/RFC4253.

[17] A. Banks, R. Gupta (Eds.), MQTT Version 3.1.1, OASIS, 2014, oASIS Standard. Available online: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html.

[18] A. Banks, E. Briggs, K. Borgendale, R. Gupta (Eds.), MQTT Version 5.0, OASIS, 2019, oASIS Standard. Available online: https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html.

[19] J. Klensin, Simple mail transfer protocol, RFC 5321, RFC Editor (October 2008). doi:10.17487/RFC5321.

[20] Caddy contributors, Caddy, release v2.11.4, June 2026.
URL https://github.com/caddyserver/caddy/releases/tag/v2.11.4

[21] stealth, sshttp, commit 91be220338249e9c7a937c8d9790f559a9fe1a60.
URL https://github.com/stealth/sshttp/tree/91be220338249e9c7a937c8d9790f559a9fe1a60

[22] Traefik contributors, Traefik, release v3.7.13, September 2026.
URL https://github.com/traefik/traefik/releases/tag/v3.7.13

[23] The Go Authors, The Go programming language, including net/http, release go1.27.1, September 2026.
URL https://github.com/golang/go/tree/go1.27.1

[24] Linux man-pages project, socket(7): Linux socket interface.
URL https://man7.org/linux/man-pages/man7/socket.7.html

[25] Linux kernel contributors, The Linux kernel source tree, tag v7.2.
URL https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tag/?h=v7.2

[26] Linux kernel documentation, BPF sk_lookup program.
URL https://docs.kernel.org/bpf/prog_sk_lookup.html

[27] Linux man-pages project, tcp(7): TCP protocol.
URL https://man7.org/linux/man-pages/man7/tcp.7.html

[28] Linux kernel documentation, BPF_MAP_TYPE_SOCKMAP and BPF_MAP_TYPE_SOCKHASH.
URL https://docs.kernel.org/bpf/map_sockmap.html

[29] The FreeBSD Project, accept_filter(9): Filter incoming connections.
URL https://man.freebsd.org/cgi/man.cgi?query=accept_filter&sektion=9

[30] Microsoft, AcceptEx function (mswsock.h).
URL https://learn.microsoft.com/en-us/windows/win32/api/mswsock/nf-mswsock-acceptex

[31] M. Lottor, TCP port service multiplexer (TCPMUX), RFC 1078, RFC Editor (November 1988). doi:10.17487/RFC1078.

[32] A. Zimmermann, W. Eddy, L. Eggert, Moving outdated TCP extensions and TCP-related documents to historic or informational status, RFC 7805, RFC Editor (April 2016). doi:10.17487/RFC7805.

[33] S. Friedl, A. Popov, A. Langley, E. Stephan, Transport layer security (TLS) application-layer protocol negotiation extension, RFC 7301, RFC Editor (July 2014). doi:10.17487/RFC7301.

[34] B. Aboba, G. Salgueiro, C. Perkins, Multiplexing scheme updates for QUIC, RFC 9443, RFC Editor (July 2023). doi:10.17487/RFC9443.

[35] M. Brinkmann, C. Dresen, R. Merget, D. Poddebniak, J. Müller, J. Somorovsky, J. Schwenk, S. Schinzel, ALPACA: Application layer protocol confusion: Analyzing and mitigating cracks in TLS authentication, in: 30th USENIX Security Symposium (USENIX Security 21), USENIX Association, 2021, pp. 4293-4310, available online: https://www.usenix.org/conference/usenixsecurity21/presentation/brinkmann.
URL https://www.usenix.org/conference/usenixsecurity21/presentation/brinkmann

[36] S. Frolov, J. Wampler, E. Wustrow, Detecting probe-resistant proxies, in: 2020 Network and Distributed System Security Symposium, Internet Society, 2020. doi:10.14722/ndss.2020.23087.

[37] S. Frolov, E. Wustrow, HTTPT: A probe-resistant proxy, in: 10th USENIX Workshop on Free and Open Communications on the Internet (FOCI 20), USENIX Association, 2020, available online: https://www.usenix.org/conference/foci20/presentation/frolov.
URL https://www.usenix.org/conference/foci20/presentation/frolov

[38] D. Poddebniak, F. Ising, H. Böck, S. Schinzel, Why TLS is better without STARTTLS: A security analysis of STARTTLS in the email context, in: 30th USENIX Security Symposium (USENIX Security 21), USENIX Association, 2021, pp. 4365-4382, available online: https://www.usenix.org/conference/usenixsecurity21/presentation/poddebniak.
URL https://www.usenix.org/conference/usenixsecurity21/presentation/poddebniak

[39] H. Dreger, A. Feldmann, M. Mai, V. Paxson, R. Sommer, Dynamic application-layer protocol analysis for network intrusion detection, in: 15th USENIX Security Symposium, USENIX Association, 2006, pp. 257-272, available online: https://www.usenix.org/legacy/event/sec06/tech/dreger.html.
URL https://www.usenix.org/legacy/event/sec06/tech/dreger.html

[40] L. Deri, M. Martinelli, T. Bujlow, A. Cardigliano, nDPI: Open-source high-speed deep packet inspection, in: 2014 International Wireless Communications and Mobile Computing Conference (IWCMC), IEEE, 2014, pp. 617-622. doi:10.1109/IWCMC.2014.6906427.

[41] L. Bernaille, R. Teixeira, I. Akodkenou, A. Soule, K. Salamatian, Traffic classification on the fly, ACM SIGCOMM Computer Communication Review 36 (2) (2006) 23-26. doi:10.1145/1129582.1129589.

[42] A. I. Tsvetanov, Bounded-wait wake defects in portable event loops, in preparation. Available online: https://github.com/Alex-Tsvetanov/paper-wake-defect (2026).

[43] Linux man-pages project, epoll(7): I/O event notification facility.
URL https://man7.org/linux/man-pages/man7/epoll.7.html

[44] Linux man-pages project, io_uring(7): Asynchronous I/O facility.
URL https://man7.org/linux/man-pages/man7/io_uring.7.html

[45] Microsoft, WSARecv function (winsock2.h).
URL https://learn.microsoft.com/en-us/windows/win32/api/winsock2/nf-winsock2-wsarecv

[46] nghttp2 contributors, nghttp2: HTTP/2 C library, release v1.70.0, July 2026.
URL https://github.com/nghttp2/nghttp2/releases/tag/v1.70.0

[47] The OpenSSL Project, OpenSSL, release 3.5.9, September 2026.
URL https://github.com/openssl/openssl/releases/tag/openssl-3.5.9

[48] A. I. Tsvetanov, A fast HTTP route matcher with compile-time checks: RegexMatcher v2 measured against sixteen routers in C, C++, Rust and Go, in preparation. Available online: https://github.com/Alex-Tsvetanov/paper-typed-routing (2026).

[49] B. Efron, Better bootstrap confidence intervals, Journal of the American Statistical Association 82 (397) (1987) 171-185. doi:10.1080/01621459.1987.10478410.

[50] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Springer US, 1993. doi:10.1007/978-1-4899-4541-9.

[51] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65-70, available online: https://www.jstor.org/stable/4615733.
URL https://www.jstor.org/stable/4615733

[52] D. J. Schuirmann, A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability, Journal of Pharmacokinetics and Biopharmaceutics 15 (6) (1987) 657-680. doi:10.1007/BF01068419.

[53] B. W. Silverman, Density Estimation for Statistics and Data Analysis, Routledge, 2018. doi:10.1201/9781315140919.